Treatments Against Sophisticated Phishing Attacks

by

Penghui Zhang

A Dissertation Prospectus Presented in Partial Fulfillment
of the Requirement for the Degree
Doctor of Philosophy

Graduate Supervisory Committee:

Gail-Joon Ahn, Co-chair
Adam Doupé, Co-Chair
Adam Oest
Alexandros Kapravelos

ARIZONA STATE UNIVERSITY

August 2020

ABSTRACT

Despite extensive research by the security community, phishing attacks remain profitable to attackers and continue to cause substantial damage not only to the victim users that they target, but also the organizations they impersonate. In recent years, phishing websites have taken the place of malware websites as the most prevalent web-based threat. Even though technical countermeasures effectively mitigate web-based malware, phishing websites continue to grow in sophistication and successfully slip past modern defenses. Phishing attack and its countermeasure have entered into a new era, where one side has upgraded their weapon, attempting to conquer the other. Therefore, the ecosystem desperately needs treatments to fight against advance phishing attacks.

In this dissertation prospectus, I design a framework, named *CrawlPhish*, to understand the prevalence and nature of such sophistications, including *cloaking*, in phishing attacks, which evade detections from the anti-phishing ecosystem by distinguishing the traffic between a crawler and a real Internet user and hence maximize the return-on-investment from phishing attacks. CrawlPhish also detects and categorizes client-side cloaking techniques in phishing with scalability and automation.

Furthermore, I will be focusing on the analysis redirection abuse in advanced phishing websites and will hence propose mitigations to classify malicious redirection use via machine learning algorithms. Based on the observations from previous work, from a totally different perspective, I will design a novel anti-phishing system that can be deployed from the user end to completely neutralize phishing attacks.

TABLE OF CONTENTS

Page

LIST OF TABLES iv

LIST OF FIGURES v

CHAPTER

1 Introduction 1

2 Background 4

2.1 Cloaking Techniques in Phishing 4

2.2 Challenges in Analyzing Client-side Cloaking 6

2.3 Redirection 7

3 Understanding and Categorizing Sophistications in Phishing 8

3.1 Crawling & Pre-processing 10

3.2 Feature Extraction 11

3.3 Cloaking Detection 13

3.4 Cloaking Categorization 15

3.5 Evaluation: Detection of Cloaked Phishing Websites 17

3.6 Evaluation: Cloaking Categorization 19

3.6.1 Categorization of Cloaking Types 20

3.6.2 Accuracy of Cloaking Categorization 24

3.6.3 Grouping of Implementations 25

3.6.4 Trends in Cloaking Usage 28

3.7 Evaluation: Impact of Cloaking techniques 30

3.7.1 Effectiveness Against Anti-Phishing Entities 31

3.7.2 Hampering Victim User Traffic 32

3.7.3 Responsible Disclosure 34

3.8 Countering Client-side Cloaking Techniques 34

CHAPTER Page

3.9 Related work 36

3.10 Conclusion 37

4 Redirection Abuse in Phishing 39

4.1 Data Collection 39

4.2 Visualization 40

4.3 Underground Services 40

4.4 Features of Malicious Use of Redirection 41

4.5 Attribute Evaluation 43

4.6 Malicious Redirection Classification 44

4.7 Related Work 45

5 Spartacus: Cloak Users Against Cloaked Phishing Websites 47

5.1 Design 47

5.1.1 Collection and Maintenance of Back End Databases 48

5.1.2 Decision Makers on A URL Visit 49

5.2 Related Work 51

6 Completed and Planned Work 53

REFERENCES 55

LIST OF TABLES

Table Page

3.1 Summary of the client-side cloaking technique types identified in this work. 9

3.2 Accuracy of cloaking detection by CrawlPhish. 18

3.3 Overview of the number of distinct groups of cloaking code implementations in the APWG and Public Datasets. 22

3.4 Cloaking technique types in the APWG Dataset, as detected by CrawlPhish. 29

3.5 Cloaking technique types in the Public Dataset (September to December 2019), as detected by CrawlPhish. 30

3.6 Top brands targeted by cloaked phishing websites in the APWG Dataset. . . . 31

3.7 Experimental results on the effect of cloaking techniques on users' ability to see phishing content. 33

4.1 Overview of redirection classifier attributes. 44

LIST OF FIGURES

1.1 The volume of phishing attacks have been surging year over year. 3

Figure Page

2.1 Typical operation of client-side cloaking in phishing websites. 5

3.1 CrawlPhish architecture. 9

3.2 ROC curves to select thresholds for cloaking detection and cloaking type categorization. 15

3.3 Initial and force-executed appearance of a phishing website with *Pop-up* cloaking. 20

3.4 A phishing website with the evolved Pop-up (Notification) cloaking technique, in which the web page directs human visitors to click on the "Allow" button by showing an arrow. 21

3.5 Appearance of a phishing website with the *Cookie* cloaking technique. 24

3.6 CDF of implementation groups for all phishing websites in the APWG Dataset. 26

4.1 Visualization of phishing websites with redirect chain. 41

4.2 Comparisons between malicious and benign redirection implementation to quantify indicators. 43

5.1 Spartacus architecture. 47

Chapter 1

INTRODUCTION

Phishing attacks have reached record levels of volume in recent years and continue to cause substantial damage—both direct and collateral—to Internet users and victim organizations. According to the analysis of Google Safe Browsing shown in Figure 1.1, phishing attacks have been increasing in recent years and have replaced malware websites to be the most popular online attack. Pandemics such as COVID-19 can also be exploited by phishers to deploy their attacks in large scale, which is observed also in Figure 1.1 as a spike in February 2020. Despite advancements in detection and mitigation efforts, the fight against phishing continues to be a cat-and-mouse game [APWG (2018); Oest *et al.* (2018)]. Sophisticated phishing websites implement evasion techniques to delay or avoid detection by automated anti-phishing systems, which, in turn, maximizes the attackers' return-on-investment [Oest *et al.* (2019)]. Such evasion—known as *cloaking*—typically seeks to determine if a visitor to the website is a bot, and shows benign content if so. The danger posed by successful evasion is exacerbated by these websites' efforts to steal more than just usernames and passwords: today's phishing attacks seek to harvest victims' full identities, which can cause wider damage throughout the ecosystem and is more challenging to effectively mitigate [Thomas *et al.* (2017)].

Thwarting phishers' evasion efforts is, thus, an important problem within the anti-phishing community, as timely detection is the key to successful mitigation. Prior research has characterized server-side cloaking techniques used by phishing websites [Oest *et al.* (2018); Invernizzi *et al.* (2016); Leontiadis *et al.* (2011)] and showed that they can defeat key ecosystem defenses such as browser-based detection [Oest *et al.* (2019)]. However, the nature and prevalence of advanced cloaking techniques, such as those implemented on the

client-side using JavaScript, is poorly understood. Client-side cloaking can be particularly dangerous because it enables the implementation of complex interactions with potential victims.

Moreover, Today's cybercrime is fueled by an underground economy that is capable of providing a full suite of commoditized services, such as web hosting [Canali *et al.* (2013a)], e-mail spamming [Stone-Gross *et al.* (2011)], and cash-out [Sood and Enbody (2013)], which greatly lowers the barrier to entry for phishing attacks and other mischief. Phishers can retrieve and exploit defaced servers from blackmarkets for malicious purposes. For this reason, servers which are defaced have subsequently been linked to future attacks such as phishing and malware [Borgolte *et al.* (2015)]. Prior work has shown a clear temporal correlation between defacements and phishing, but prediction been limited in scope to phishing sites with the same domain as the defacement [Wardman *et al.* (2016)] or *existing* web pages which subsequently turn malicious [Soska and Christin (2014)].

The ultimate desire of phishers is to maximize the profits from attacks by extending the lifespan of their phishing websites as long as possible and to lure real human visitors to submit credentials. If we compare phishing attacks with COVID-19, techniques such as cloaking, redirection, and re-use of defaced servers are the symptoms as cough, fever, and shortness of breath caused by the virus. To eliminate COVID-19 from human's body, vaccines are needed and take in effect inside a human body to neutralize the coronavirus. Similarly, to destroy phishing attack completely, we need an anti-phishing system that neutralize phishing content from the user end.

Therefore, in this dissertation prospectus, I propose both pain relief pills and a vaccine against phishing-virus as follows. I design a framework named *CrawlPhish* to first understand the nature and prevalence of client-side cloaking techniques in phishing and then to detect and categorize cloaked phishing websites automatically. I then analyze the redirection abuse in phishing evading detection from anti-phishing systems and design a

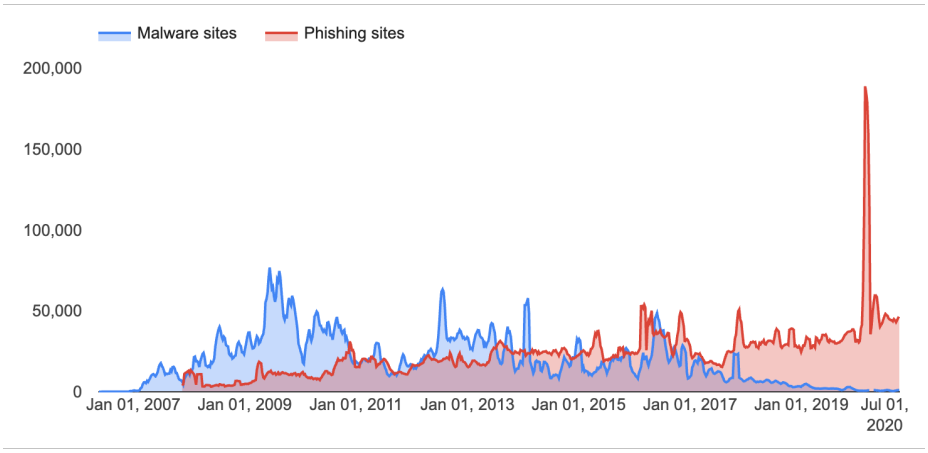


Figure 1.1: The volume of phishing attacks have been surging year over year.

classifier using machine learning algorithms to discover malicious websites using redirection attributes. Lastly, during the following of my Ph.D. career, I will design a vaccine, namely *Spartacus*, to cure the disease of phishing attacks by neutralizing phishing content from the user end. Without seeing impersonated websites, potential victims will never submit the credentials.

Chapter 2

BACKGROUND

Over the past years, a myriad of techniques have been implemented by the anti-phishing ecosystem to detect and mitigate phishing attacks [Oest *et al.* (2018)]. Analysis of phishing URLs [Bin *et al.* (2010); Blum *et al.* (2010); Huang *et al.* (2012); Khonji *et al.* (2011)] and website content [Wu *et al.* (2006); Zhang *et al.* (2007, 2011a); Bilge *et al.* (2011); Canali *et al.* (2013b)] has given rise to ecosystem-level defenses such as e-mail spam filters, malicious infrastructure detection, and URL blacklists.

Specifically, systems such as Google Safe Browsing [Whittaker *et al.* (2010)] and Microsoft SmartScreen [Microsoft (2019)] power the anti-phishing backends that display prominent warnings across major web browsers when phishing is detected. These warnings are primarily blacklist-based: they rely on content-based detection. Evasion techniques commonly used by phishing websites are capable of bypassing or delaying such blacklisting [Liang *et al.* (2016); Oest *et al.* (2019, 2020b)].

2.1 Cloaking Techniques in Phishing

Attackers leverage *cloaking techniques* to evade detection by anti-phishing systems: phishing websites with cloaking display benign-looking content instead of the phishing page whenever they suspect that a visit originates from security infrastructure [Oest *et al.* (2018)]. Server-side cloaking techniques identify users via information in HTTP requests [Wang *et al.* (2011)]. Client-side cloaking is implemented through code that runs in the visitor's browser (JavaScript) to apply filters using attributes such as cookies or mouse movement.

Existing anti-cloaking methodologies focus on bypassing server-side cloaking by comparing the visual and textual features of different versions of a crawled website retrieved

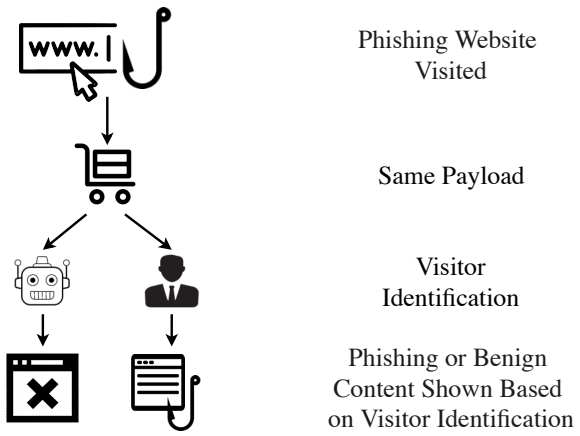


Figure 2.1: Typical operation of client-side cloaking in phishing websites.

by sending multiple web requests with different configurations (e.g., user agents or IP addresses) [Guo (2017); Wang *et al.* (2011); Invernizzi *et al.* (2016)]. Client-side cloaking techniques, however, are still poorly understood due to challenges in automatically analyzing JavaScript code and understanding its semantics. Moreover, neither the prevalence nor impact of client-side cloaking has been investigated in the context of phishing.

Figure 2.1 shows how client-side cloaking techniques are used in phishing websites. Cloaking code embedded in the HTTP response payload shows different web page content based on the identification of visitors (as either potential victims or bots). Consequently, cloaked phishing websites may have a longer life span than ones without: by delaying or avoiding detection, the attackers who launch these websites maximize their return-on-investment [Oest *et al.* (2019)]. Because client-side evasion techniques enable complex interactions between potential victims and phishing websites, they may be more effective in hampering automated detection than traditional server-side cloaking, and, thus, pose a threat to potential victim users.

2.2 Challenges in Analyzing Client-side Cloaking

Unlike server-side code, the client-side code (JavaScript) of websites can trivially be obtained through crawling. Therefore, malicious websites typically leverage code *obfuscation* methods such as string array encoding, object key transformation, dead code injection, and even full encryption [Cova *et al.* (2010); Kachalov and zamotkin (2016)]. Attackers also can dynamically generate and execute code (e.g., using `eval`) to hide malicious behaviors. Such obfuscation methods pose a challenge for static code analysis approaches, which are otherwise favored for their efficiency.

Other types of obfuscation also seek to prevent dynamic analysis approaches from detecting malicious behaviors. Malicious JavaScript code often targets specific versions of web browsers and operating systems by *fingerprinting* them [Curtsinger *et al.* (2011)]. Such attacks are difficult to discover because detection systems require extensive resources to reveal the conditions that trigger attacks [Cova *et al.* (2010)]. Besides, external and inter-block *dependencies*, which require recording states in different execution paths, can be obstacles that thwart the analysis of JavaScript code [Kim *et al.* (2017)]. Furthermore, scripts may execute in an *event-driven* manner to necessitate external triggers to initiate malicious behavior while otherwise appearing benign [Kim *et al.* (2017)].

All of the aforementioned anti-analysis methods can potentially be leveraged by phishing websites' implementations of client-side cloaking techniques. Given the difficulty of analyzing such cloaking, the security community struggles to thoroughly understand the impact and prevalence of phishers' tactics, and, thus, may fail to appropriately mitigate them. When we consider the scale on which phishing attacks occur [APWG (2019)], the consequences of the corresponding gaps in detection and mitigation can be significant.

2.3 Redirection

Redirection refers to an activity that when a web browser attempts to visit a URL that has been redirected, a web page with a different URL is opened. The implementation of redirect can be through server (e.g., 3XX status code) or in the client browser (JavaScript and meta tag in HTML). Redirection contains both benign and malicious usage. People implement redirection to shorten long URL links, prevent broken links, protect privacy, and etc. However, miscreants (phishers) leverage such technique to evade detection from anti-phishing systems. Criminals often shorten the phishing URLs spread in social media so that ordinary users may not notice whether the link is legitimate [Aggarwal *et al.* (2012)]. They also exploit the open redirect vulnerabilities found in Google, YouTube, Google Docs, and etc., where a legitimate website allow any redirections, to bypass the URL based anti-phishing systems [Stockley (2020)]. Last but not the least, phishers implement redirect chains, where more than one redirection occurs from the entry URL to the destination [Kwon *et al.* (2017)]. As researchers proposed several mechanisms to detect and mitigate malicious use of redirection in phishing, the miscreants began to embed advanced evasion techniques such as cloaking techniques to power the redirection.

Chapter 3

UNDERSTANDING AND CATEGORIZING SOPHISTICATIONS IN PHISHING

Client-side cloaking techniques can help phishing websites evade detection by anti-phishing entities [Oest *et al.* (2019)], yet prior studies have not investigated them in detail, despite evidence that sophisticated phishing websites—such as those with client-side cloaking—are responsible for a majority of real-world damage due to phishing [Oest *et al.* (2020c)].

I discover eight different types of JavaScript cloaking techniques across three high-level categories: User Interaction, Fingerprinting, and Bot Behavior (summarized in Table 3.1). Cloaking techniques in the *User Interaction* category show phishing content only if visitors interact with a phishing website (e.g., by moving the mouse or clicking a specific button). Phishing websites with *Fingerprinting* identify visitors by inspecting the configuration of browsers or web requests. Finally, phishing websites with *Bot Detection* identify anti-phishing crawlers based on factors such as how long the web page stays open and whether the web request is repeated after failing initially.

I aim to comprehensively understand and characterize the landscape of client-side cloaking techniques used by phishing websites in the wild through an automated methodology for analyzing them. To this end, I design, implement, and evaluate *CrawlPhish*: a framework that automatically detects and analyzes client-side cloaking within phishing websites. Figure 3.1 provides an overview of the CrawlPhish architecture. CrawlPhish is composed of the following components:

① **Crawling and pre-processing (§3.1):** CrawlPhish first collects web page source code (along with any external file inclusions) by visiting live phishing website URLs recently reported to anti-phishing feeds. We then filter URLs that cannot be retrieved as well as

Cloaking Category	Cloaking type	Requirement
User Interaction	Pop-up	Click on alert/notification window
	Mouse Detection	Move mouse over browser
	Click Through	Pass Click Through on browser
Fingerprinting	Cookie	Check document.cookie
	Referrer	Check document.referrer
	User-Agent	Check navigator.userAgent
Bot Behavior	Timing	Render webpage after certain time using sleep()/Date.getTime()
	Randomization	Show content randomly using Math.random()

Table 3.1: Summary of the client-side cloaking technique types identified in this work.

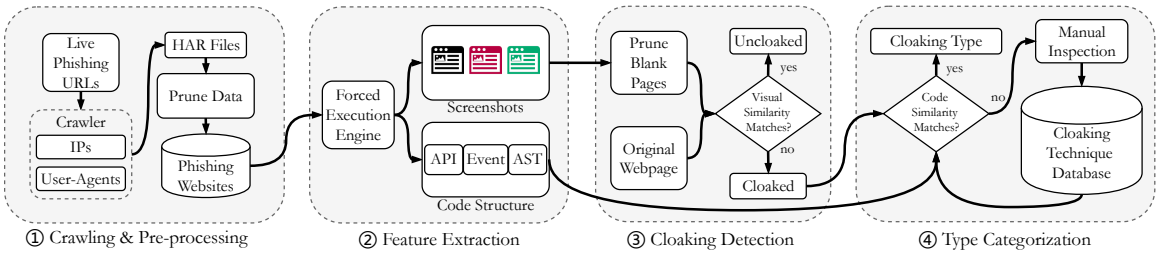


Figure 3.1: CrawlPhish architecture.

URLs without any JavaScript code.

② **Feature extraction (§3.2):** CrawlPhish adapts a state-of-the-art code analysis method, *forced execution* [Kim *et al.* (2017)], to execute JavaScript regardless of branch conditions, and extracts all possible execution paths in which evasion techniques could be implemented. We then derive (1) visual features of the rendered web pages, by means of screenshots, and (2) code structure features such as web API calls, event listeners, and the Abstract Syntax Tree (AST) for each path.

③ **Cloaking detection (§3.3):** CrawlPhish analyzes the visual features corresponding to each execution path to detect if cloaking exists, and it stores the corresponding code structure features of every such path.

④ **Cloaking categorization (§3.4):** Using the code structure features, CrawlPhish categorizes the cloaking techniques used by phishing websites based on their semantics.

3.1 Crawling & Pre-processing

To collect the source code of *live* phishing websites to detect and classify client-side evasion methods that are currently employed in the wild, CrawlPhish first obtains URLs of known phishing websites in real-time.

In our deployment, CrawlPhish continuously ingested URLs from the APWG eCrime Exchange database—a curated clearinghouse of phishing URLs maintained by various organizations engaged in anti-phishing. Because this database receives frequent updates and tracks phishing URLs that target a diverse range of brands, it is well-suited for phishing website analysis. [1] Note, however, that the inclusion of a URL in the database does not mean that it was adequately mitigated (e.g., through timely blacklisting) [Oest *et al.* (2020b)]. Hence, websites found to use sophisticated client-side cloaking still warrant scrutiny.

Next, CrawlPhish downloads source code by visiting each phishing website URL (shortly after being ingested) using a programmatically controlled web browser. Specifically, CrawlPhish stores source code using HAR files, which capture all HTTP requests/responses between our client and the server, and ensure that all dependencies (such as linked scripts) are preserved for each website. In case of a failed request, CrawlPhish switches between different configurations of IP addresses and user-agents in an effort to circumvent potential server-side cloaking techniques used by phishing websites [Oest *et al.* (2018)]. 4,823 of the 128,973 websites we crawled (3.74%) showed different response status codes after we switched request configurations.

[1] Although the goal of cloaking is to evade detection by automated anti-phishing systems, such evasion will often delay detection rather than outright prevent it. Phishing websites may also be detected by other means (e.g., manual review) Oest *et al.* (2020c). Thus, we expected the AWPG database to contain a representative sampling of any client-side cloaking that might be used in the wild.

Finally, CrawlPhish filters out URLs that contain blank pages or non-phishing websites. Such websites were either already taken down [Alowaisheq *et al.* (2019)] or were false-positive detections by the time of crawling. We found 0.53% of URLs within the APWG Dataset to be false positives. Therefore, CrawlPhish excludes data in the following cases:

i. *empty* websites: servers respond with no content.

ii. *error* websites: requests for URLs were denied because the phishing websites were already taken down, or used server-side cloaking which we could not bypass.

iii. *non-phishing* websites: mistakenly reported URLs, which CrawlPhish filters based on a manually curated whitelist of reputable domains.

3.2 Feature Extraction

Cloaked content detection. Client-side cloaking techniques used in phishing websites can be more diverse than server-side cloaking because they can not only fingerprint visitors based on configurations of browsers and systems, but may also require visitors to interact with websites. To effectively detect client-side cloaking techniques, CrawlPhish adapts J-Force: a *forced execution* framework implemented in the WebKitGTK+ browser that executes JavaScript code along all possible paths, crash-free, regardless of the possible branch conditions, event handlers, and exceptions Kim *et al.* (2017). We modified J-Force to whitelist (avoid force-executing) well-known JavaScript libraries, such as *Google Analytics* or *jQuery*, to expedite execution by ignoring the benign content changes that such libraries could introduce.

Execution time limit. We select a time limit for each invocation of forced execution by CrawlPhish to avoid failures due to long-running scrips (e.g., due to heavy branching or long-running loops).

As a starting point, we chose an execution limit of 300 seconds. We conducted an experiment by force-executing 2,000 randomly selected phishing websites in our crawled dataset to record the execution time. We found that 1.75% of phishing websites contained JavaScript code that exceeded the time limit. Execution finished as quickly as 12.56 seconds, the median execution time was 13.82 seconds, the average execution time was 29.96 seconds, and the standard deviation was 54.89 seconds. Based on this experiment, we chose a final execution limit of 195 seconds (three standard deviations above the mean) so that CrawlPhish could efficiently analyze the majority of phishing websites.

Feature extraction. To facilitate detection of (the existence of) cloaking and categorization of the corresponding cloaking type, CrawlPhish extracts both *visual* and *code structure* features from each phishing website. Each phishing website's visual features consist of the set of all web page screenshots (in our implementation, at a resolution of 2,495$\times$1,576 pixels) captured after every possible execution path is explored by forced execution. In our dataset, each website generated 46.3 screenshots on average. CrawlPhish compares the screenshots of *each* execution path within one website against the original screenshot to detect if cloaking exists, because the presence of cloaking will result in significant visual layout changes Wang *et al.* (2011). The code structure features include web API calls, web event listeners, and ASTs, which can characterize different types of cloaking techniques and reveal how the cloaking techniques are implemented. Using forced execution, CrawlPhish can reveal and extract the web APIs and events contained in every code block, even if the code is obfuscated. CrawlPhish can then classify the cloaking types in a website using the code structure features.

Code structure features used. According to preliminary analysis which we conducted by manually inspecting cloaking techniques in a sampling of phishing websites in our dataset, different client-side cloaking techniques each have substantially different features. For example, a cloaking technique that checks mouse movement waits for an `onmousemove`

event, then performs DOM substitution or redirection. However, a cloaking technique that checks screen size would first access the `screen.height` property. Therefore, as CrawlPhish executes a code block via forced execution, it records the web APIs and events that are invoked in the code block.

In addition, we found that the same semantic types of client-side cloaking techniques have many different implementations. CrawlPhish distinguishes between different implementations of each type of cloaking technique by comparing ASTs. Even though JavaScript code is often obfuscated, the AST feature is still useful because most phishing websites are deployed using phishing kits, so the corresponding websites, with the same phishing kit origin, share the same source code structure Thomas *et al.* (2017). Furthermore, by computing the AST similarity, we can trace the origin of the cloaking technique by finding similar implementations earlier in phishing pages.

3.3 Cloaking Detection

CrawlPhish examines the *visual similarity* between force-executed screenshots and a screenshot of the website rendered in an unmodified version of WebKitGTK+ (i.e., as would be shown during a normal browser visit) to detect if cloaking exists. Because phishers implement JavaScript cloaking techniques to *evade* detection by anti-phishing systems, they remove suspicious attributes in websites (e.g., login forms) or outright redirect to a benign website. Therefore, the visual content shown when the cloaking condition is not satisfied will differ significantly from that of the malicious page.

For example, consider a phishing website that asks visitors to click on a button in a pop-up window prior to showing the phishing content. After forced execution, two different execution paths result in two different screenshots: one as an initial benign-looking page (Figure 3.3a), and the other with phishing content (Figure 3.3b). Therefore, we consider a phishing website as *cloaked* if any of the screenshots taken during forced execution

noticeably differ from the original one.

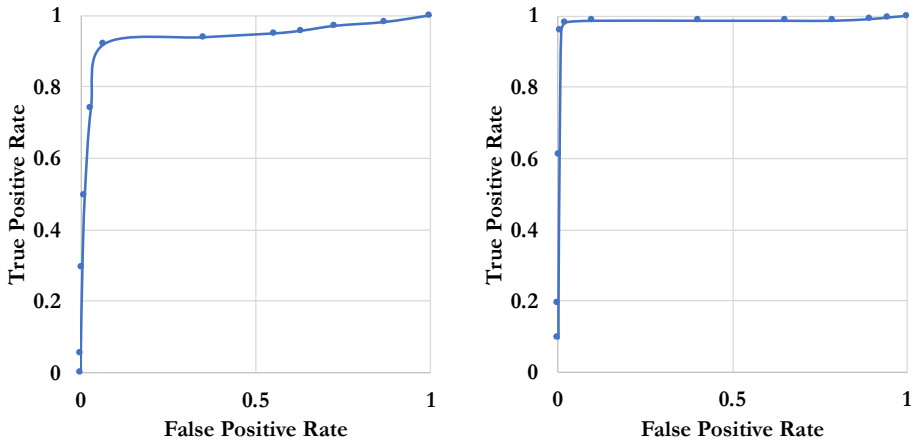
CrawlPhish can also reveal phishing content hidden behind multiple layers of cloaking. Consider a phishing website with a cloaking technique that (1) detects mouse movement and (2) checks the referrer such that the malicious content will appear only if both requirements are met. CrawlPhish will explore the execution path that shows the malicious content by force-executing it, regardless of the branching conditions. Therefore, after each screenshot is compared with the screenshot of the original page, CrawlPhish determines that a cloaking technique exists because one of the screenshots will differ.

Removal of blank pages after forced execution. Screenshots of pages rendered by force-executed paths may be blank, which can be caused by (1) negative branches from cloaking techniques (such as mouse movement detection) that require user input or (2) execution paths that take longer to finish than the execution time limit. In the latter case, CrawlPhish can mislabel a website as *cloaked* if an initial screenshot is compared to an empty page caused by unfinished execution paths. For example, phishers may trigger an infinite loop if they identify that a visit is from an anti-phishing system. In this case, CrawlPhish cannot finish forced execution and hence the screenshot remains empty. Thus, a current limitation of CrawlPhish is that it cannot detect cloaked websites with very long execution times, which we explain in **??**. However, according to our evaluation, this situation does not happen often: only in 1.75% of the websites we considered.

Detection algorithm. To perform visual similarity checks between the screenshots, we implement the pHash algorithm Niu and Jiao (2008), which compares visual similarity with robustness and good discrimination. We calculate pHash scores between the original screenshot and those captured after each path finishes execution.

$$score = pHash(S_{original}, S_i), i \in [1, 2, ..., n] \tag{3.1}$$

In Formula 1, S represents each screenshot and n is the number of screenshots captured



(a) Visual feature threshold. (b) Code structure feature threshold.

Figure 3.2: ROC curves to select thresholds for cloaking detection and cloaking type categorization.

from forced execution. We consider two screenshots to be similar (no cloaking) if the pHash score is less than a threshold (5.0) that we set based on preliminary testing results on 1,000 phishing websites. Differing screenshots will have a score of 5.0 or greater. Figure 3.2a shows the ROC curve for selecting the visual similarity threshold. We selected the threshold that provides a 92.00% true-positive rate with a 6.77% false-positive rate. We note that our evaluation in section 3.5 shows that CrawlPhish exhibited higher detection accuracy (98.25%) with a lower false-positive rate of 1.45% than what was indicated by the threshold in the ROC curve.

3.4 Cloaking Categorization

Once CrawlPhish detects the *presence* of cloaking on a web page, categorization of the specific *type* of cloaking allows us to measure and understand the prevalence of different high-level client-side cloaking techniques used by phishers. To facilitate this categorization, CrawlPhish maintains a *cloaking technique database* that contains the code structure

features for each instance of cloaking, annotated with the corresponding cloaking semantics. Using the database, CrawlPhish can not only identify known cloaking types, but also provide detailed information about emerging cloaking techniques.

Initial database. We first obtained 1,000 cloaked phishing websites (true positives), for which we used CrawlPhish to determine the existence of client-side cloaking. Then, we manually examined the source code of the phishing websites to label the corresponding cloaking techniques. We also recorded code structure features as ground truth.

For example, we labeled one type of cloaking technique as *Mouse Detection* if the recorded code features have the `onmousemove` event and use the `window. location.href` API. Over time, as CrawlPhish executes, if the presence of cloaking is detected on a website but the code features *do not* sufficiently closely match any of the records in the database, the website is flagged for manual review such that the missing features (and, potentially, new cloaking types) can be populated. Otherwise, the website is automatically labeled with the corresponding semantic cloaking type. Within the dataset we crawled, manual effort was rarely needed after we populated the initial database. Thus, this requirement does not impede the automated operation of our framework.

Categorization algorithm. CrawlPhish employs the Hamming Distance (HD) algorithm Hamming (1950) to compute the similarity of the API calls and web events. To this end, we use an array data structure with one position for each of the 4,012 types of web API calls or events as defined by the Mozilla MDN Docs (2020); Mozilla (2020), which documents currently available web APIs. At each position in the array, we store the number of corresponding API calls or events as observed by CrawlPhish. We then convert this array to a fixed-length string (e.g., string[0] is the number of `ActiveXObject` in the code block and string[1] stores the amount of `Date` API calls) so that we can apply the HD algorithm. Thus, the result of the HD algorithm on a pair of strings represents the similarity of web APIs and events between two code blocks. Lower HD values indicate higher similarity.

We also leverage JSInspect Jules (2017) to find structurally similar code snippets based on the AST. This will identify code with a similar structure based on the AST node types (e.g., BlockStatement, VariableDeclaration, and ObjectExpression). We combine these approaches to overcome limitations of code similarity checkers based solely on either ASTs or API calls. Consequently, by comparing the code structure similarity of all suspicious code blocks against records in the database, all known cloaking types can be identified in one website (even if there are multiple types). If the features of a suspicious code block are not sufficiently similar to any record in the database, we will manually examine it, label the cloaking type, and then add it to the database, which is the only process that requires manual effort in the CrawlPhish framework.

Similar to the visual similarity check, we empirically set a threshold for the code similarity check based on preliminary manual analysis of 1,000 cloaked phishing websites. We consider only two categories to find a threshold: correctly labeled cloaking types and mislabeled cloaking types. Per Figure 3.2b, we selected a code structure threshold with a true-positive rate of 95.83% and a false-positive rate of 0.79%. When CrawlPhish compares the code structure features of a new phishing website to ones in our database, the AST similarity score must be greater than 0.74 *and* the Hamming Distance of web APIs and events must be within 34 for a new website to be marked with a known type of cloaking technique.

3.5 Evaluation: Detection of Cloaked Phishing Websites

In this section, we evaluate the client-side cloaking detection accuracy of CrawlPhish. In this experiment, we first randomly sampled and manually labeled 2,000 phishing websites that did not contain JavaScript cloaking techniques as well as 2,000 phishing websites with various types of client-side cloaking. We then ran CrawlPhish to detect if client-side cloaking exists. Finally, we compared the automated cloaking detection results against our

Crawled Phishing Websites From APWG		Analyzed			
		Cloaked		**Non-cloaked**	
Actual	**Cloaked**	TP		FN	
		1,965	98.25%	35	1.75%
	Non-cloaked	FP		TN	
		29	1.45%	1,971	98.55%

Table 3.2: Accuracy of cloaking detection by CrawlPhish.

manually labeled ground truth dataset to calculate the detection accuracy.

Table 3.2 shows the confusion matrix of CrawlPhish's detections. Within the 4,000 phishing websites, CrawlPhish correctly detected 1,965 phishing websites as cloaked and 1,971 as uncloaked, with a false-negative rate of 1.75% (35) and a false-positive rate of 1.45% (29). Note that unlike a general phishing detection tool that should prioritize false positives over false negatives Whittaker *et al.* (2010), the client-side cloaking detection component in CrawlPhish does *not* critically need to do so, because the goal of our detection is to study the nature of client-side cloaking, rather than to detect a phishing attack. If CrawlPhish trades higher false negatives for lower or even zero false positives, the study might be less complete because we might miss many relevant instances of cloaking. Therefore, the detection of CrawlPhish should balance false positives with false negatives.

Each of the 29 false-positive cases was caused by one of two errors. The first error was due to the rending overhead of the unmodified browser which loaded the original phishing page. WebKitGTK+, the web browser we used in the CrawlPhish framework, failed to render the original websites within an allotted time limit due to a large number of CSS and JavaScript files included by the website. As a result, the original screenshot of each website was blank, but the screenshots after forced execution were not blank, so CrawlPhish mislabeled the corresponding websites as *cloaked* because the screenshots differed before and after forced execution. The second error was caused by inaccuracies in our image

similarity checks. The image similarity check module erroneously distinguished between screenshots of identical pages due to slight variations in the page layout generated by the browser with and without forced execution.

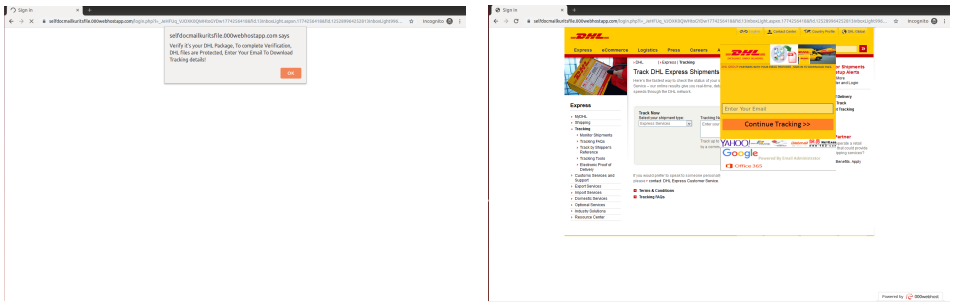
In terms of the false negatives, we found that 32 out of the 35 stemmed from a long execution time of cloaked phishing websites (similar to the first reason for false positives). Forced executed screenshots are not taken if an execution path takes too long to finish execution. We used a 195-second execution time window for each execution path. However, the paths that CrawlPhish does not execute due to a timeout may contain cloaking technique implementations. Without those screenshots, CrawlPhish cannot detect the cloaking technique, so it mislabels the corresponding website as *uncloaked*.

In three rare cases, real phishing websites appeared nearly blank due to low page contrast. For example, if phishing websites have a white background with light text, CrawlPhish would not distinguish between the corresponding screenshot and a blank one. We manually examined these cases and found that CSS inclusions were missing from those websites (i.e., they could not be retrieved by our crawler).

Client-side cloaking occurrence statistics. Within our dataset of 112,005 phishing websites, CrawlPhish found that 35,067 (31.31%) phishing websites implement client-side cloaking techniques in total: 23.32% (6,024) in 2018 and 33.70% (29,043) in 2019. We note that cloaking implementations in phishing grew significantly in 2019. We hypothesize that phishers are either leveraging such cloaking because it increases their profitability or because improving detection systems make advanced evasion necessary, or both.

3.6 Evaluation: Cloaking Categorization

In this section, we elaborate on the eight types of client-side cloaking techniques detected by CrawlPhish (as previously introduced in Table 3.1). We also evaluate the accuracy of CrawlPhish's semantic cloaking categorization, track trends in the deployment



(a) Initial appearance. (b) Force-executed appearance.

Figure 3.3: Initial and force-executed appearance of a phishing website with *Pop-up* cloaking.

and evolution of different implementations of these cloaking techniques, and analyze how frequently they are used.

3.6.1 Categorization of Cloaking Types

User Interaction: *Pop-up*. With this technique, phishing content remains hidden until a button in a pop-up window is clicked. Specifically, JavaScript code listens for an `onclick` event to evade anti-phishing bots. Figure 3.3 shows an example of a phishing website that implements this technique. The website in Figure 3.3a initially shows an `alert` window to an anti-phishing bot or a real user. Thus, this phishing website seeks to evade detection by anti-phishing bots because no phishing content or typical attributes (such as a login form or logos of a legitimate organization) are found on the page. However, CrawlPhish reveals the phishing content hidden behind the popup window as shown in Figure 3.3b.

Figure 3.4 shows a more advanced version of the pop-up cloaking techniques that CrawlPhish detected. Because an `alert` window can easily be closed through common browser automation frameworks such as Selenium Holmes and Kellogg (2006) or Katalon Katalon (2015), some phishers instead use the *Web Notification API* W3C (2015). We observed that due to technical limitations, top automation frameworks Anderson (2017) do

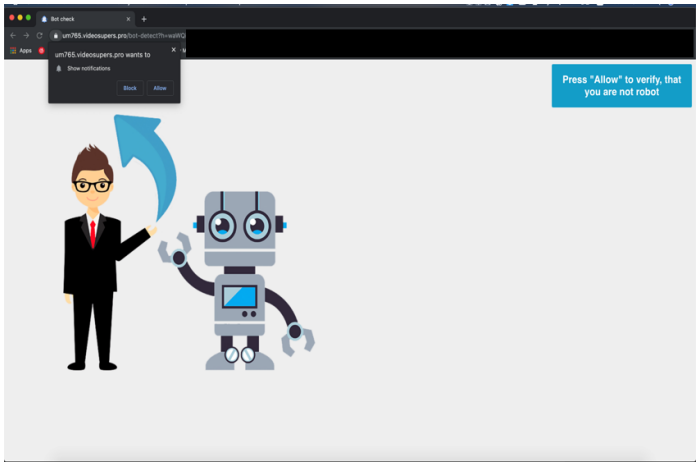


Figure 3.4: A phishing website with the evolved Pop-up (Notification) cloaking technique, in which the web page directs human visitors to click on the "Allow" button by showing an arrow.

not currently support interaction with web notifications. These automated browsers opt to disable the notification window to avoid such interactions. Phishers, however, only allow visitors who actually click the "Allow" button to access the phishing content. Therefore, because the phishing website will not show any phishing content until a visitor clicks the "Allow" button in the notification window, it will evade detection. Phishers use a deceptive web page that asks visitors to click the button on the notification window, as shown in Figure 3.4. As an added benefit to attackers, by using a notification window, cloaked phishing websites could also directly send spam to visitors through their browsers (we do not evaluate the extent of such abuse). Through this, we show that criminals are using cutting-edge browser features to evade existing detection systems.

User Interaction: *Mouse Detection*. This cloaking type seeks to identify whether a website visitor is a person or an anti-phishing bot by waiting for mouse movement before displaying the phishing content. Specifically, the cloaking code listens for the `onmousemove`, `onmouseenter`, or `onmouseleave` events. This technique is used frequently by phish-

Cloaking Technique			Public Dataset			APWG Dataset					Identical Groups
			Unique	Top Group		Unique	Top Group		Earliest	Groups Used	
Category	Type		Groups	Count	Percentage	Groups	Count	Percentage	Impl.	From 2018	
Fingerprinting	Cookie		43	437	15.01%	28	325	7.39%	09/2018	12	14
	Referrer		27	156	5.85%	37	92	3.92%	08/2018	21	9
	User-Agent		65	563	53.31%	33	181	12.97%	07/2018	24	20
User Interaction	Pop-up	Alert	424	249	3.26%	335	73	1.21%	06/2018	276	127
		Notification	29	52	4.22%	17	284	18.67%	11/2018	7	11
	Click Through		105	1,541	22.88%	51	1,275	16.45%	10/2018	13	31
	Mouse Detection		87	138	6.81%	108	500	8.63%	06/2018	47	37
Bot Behavior	Randomization		73	42	16.03%	125	58	3.57%	09/2018	62	43
	Timing		597	387	7.76%	394	416	5.99%	06/2018	303	197

Table 3.3: Overview of the number of distinct groups of cloaking code implementations in the APWG and Public Datasets.

ers, and accounts for 16.53% of all cloaking technique implementations in Table 3.4, because most people have a habit of moving the mouse while a website is rendering in the browser Rotolo (2016).

User Interaction: *Click Through*. Some phishing websites require visitors to click on a specific location on the page before displaying phishing content Wang and Ma (2006). Simple variants of this cloaking technique require visitors to click on a button on the page and are, thus, similar to `alert` cloaking. However, more sophisticated variants display fake CAPTCHAs that closely mimic the look and feel of Google's reCAPTCHA Von Ahn *et al.* (2008). Given the common use of reCAPTCHA by legitimate websites, phishing websites with fake CAPTCHAs make it difficult for potential victims to identify that they are fake. If anti-phishing systems cannot access phishing content because of the Click Through technique, they may fail to mark the websites as phishing.

Bot Behavior: *Timing*. Some phishing websites show phishing content only at a certain time, or deliberately make rendering slow by using the `setTimeout()` or `Date.getTime()` APIs. If phishing websites take a longer time to render than thresholds set by detection

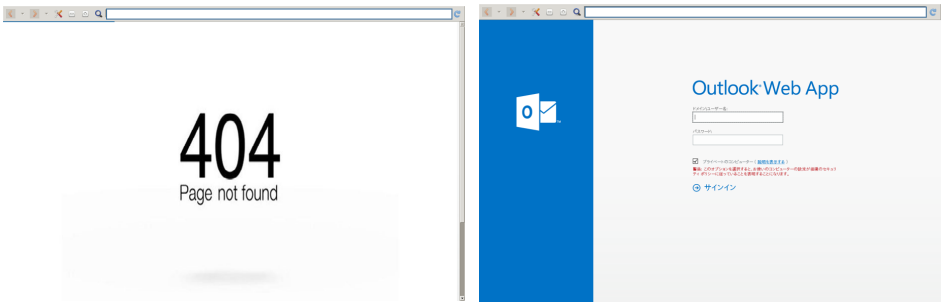
systems, such websites can evade detection. Actual visitors, however, might wait for the completion of web page rendering Dhamija *et al.* (2006).

Bot Behavior: *Randomization*. Some phishers try to evade detection by using a non-deterministic mechanism: such phishing websites generate a random number before the page is rendered, and only show phishing content if a certain threshold is met. Anti-phishing crawlers or human inspectors may not visit the same website again if it initially shows benign content. Therefore, this technique may appear to be a "dumb" way to evade detection by anti-phishing systems. However, its use in the wild suggests that it may be worthwhile: we suspect that phishers who use this technique are aware of the conditions for detection by anti-phishing entities and try to trick anti-phishing bots with a non-deterministic approach to cloaking.

Fingerprinting: *Cookie*. Similar to server-side cloaking techniques, client-side cloaking techniques can also check visitors' request attributes to fingerprint them. Figure 3.5 illustrates a phishing website that fingerprints whether a visitor is a person or an anti-phishing bot by checking if cookies are disabled in the browser. When cookies are disabled, the phishing websites will display benign content, as shown in Figure 3.5a. Some anti-phishing crawlers disable cookies to avoid being bound to a single session. However, CrawlPhish detects cloaked phishing content as shown in Figure 3.5b. Similarly, this cloaking technique may also test if the *browser cache* is enabled Paya and Chow (2007).

Fingerprinting: *Referrer*. Phishing websites can check whether incoming traffic originates from phishers' lures or other unwanted sources. Therefore, some phishing websites display benign content to visitors with a blank *Referer* Fielding *et al.* (1999), which could indicate that a URL was directly typed in. Similarly, referrals from search engines or known security domains can be blocked.

Fingerprinting: *User-agent*. Some phishing websites seek to identify anti-phishing crawlers based on their user-agent strings. The `navigator.userAgent` property stores infor-



(a) Benign page shown when cookies are disabled.

(b) Force-executed version, which reveals the login form.

Figure 3.5: Appearance of a phishing website with the *Cookie* cloaking technique.

mation about the browser and operating system (e.g., Mozilla/5.0 (X11; Linux x86_64)). Therefore, anti-phishing bots such as *Googlebot* can be blocked as their `userAgent` property is a known value.

Combinations of cloaking techniques. Multiple client-side cloaking techniques are occasionally used together by phishing websites, as doing so may further increase evasiveness. For example, CrawlPhish found 503 instances of *Click Through* and *Referrer* used together. Also, we found *Timing* and *Cookie* in 476 cloaked phishing websites.

3.6.2 Accuracy of Cloaking Categorization

To evaluate the accuracy of CrawlPhish's categorization of cloaking types, we selected the same 2,000 cloaked phishing websites as in section 3.5 (this set contains all three categories of client-side cloaking techniques) and manually labeled the correct cloaking type based on their code structure features. We, then, sent these websites through the feature extraction (②) and the cloaking detection (③) phases of CrawlPhish to locate the code blocks in which each cloaking technique is implemented. CrawlPhish checked the code structure feature similarity as populated over the course of our deployment (④). As stated in section 3.4, CrawlPhish compares the code structure features of all snippets flagged by

Step ③ with the records in the database to discover all possible cloaking techniques in a given phishing website.

We found that CrawlPhish correctly categorized the cloaking type with 100% accuracy. This high accuracy stems in part from the manual inspection involved when the code structure features of the examined snippet do not match any existing records in the database, as discussed in section 3.4. Thus, we conclude that web API calls, web events, and ASTs suffice for distinguishing between different cloaking types, even when the underlying implementations vary.

3.6.3 Grouping of Implementations

Because phishing kits directly enable the scalability of phishing attacks and are readily available through underground markets Sun *et al.* (2019); Thomas *et al.* (2017); Modi *et al.* (2016), tracking the deployment and evolution of kits can help researchers and investigators pinpoint the threat actor (i.e., a kit author or criminal group) behind a series of phishing websites and identify the prevalence phishing attacks attributable to the same author. The web page source code collected by CrawlPhish is suitable for this purpose because such source code can be obtained for virtually any phishing URL—unlike server-side code Oest *et al.* (2018).

By comparing code similarity between JavaScript snippets used by cloaked phishing websites, over time, we can group related cloaking technique implementations (i.e., implementations attributable to the same origin) together. Specifically, we compare the AST similarity among cloaking technique implementation code blocks to find matches using JSInspect Jules (2017) (the same technique we leveraged to check the code structure similarity). In Table 3.3, we provide an overview of the number of implementation groups that we found for each cloaking technique within the APWG Dataset and the Public Dataset. In addition, we compare the overlap in groups between the two datasets, and we determine

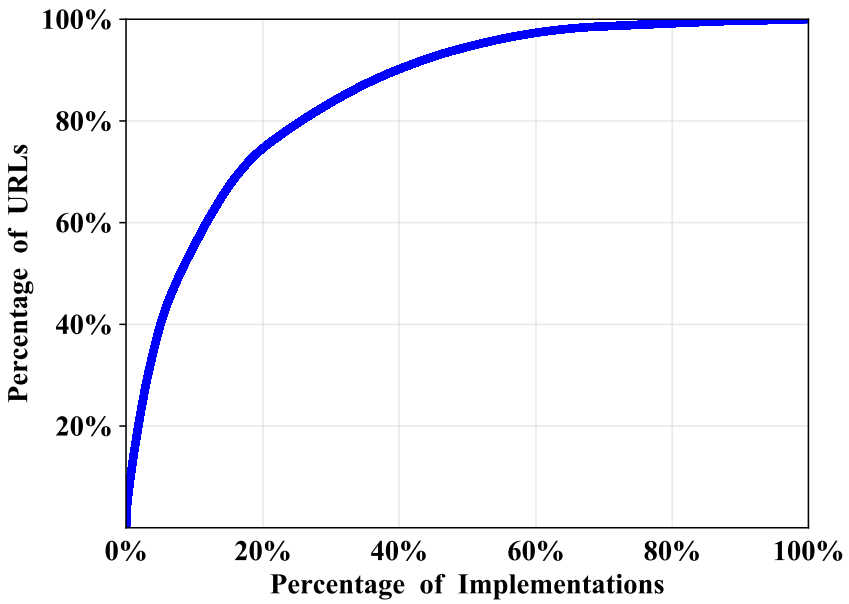


Figure 3.6: CDF of implementation groups for all phishing websites in the APWG Dataset.

the earliest date that each technique was observed.

Implementation groups in the APWG Dataset. We found that the earliest implementation of *each* cloaking type was in 2018. Also, we found that 1,128 groups account for 35,067 cloaked phishing websites detected by CrawlPhish. Figure 3.6 shows the cumulative distribution function (CDF) of unique implementation groups in the APWG Dataset: 20% of unique cloaking implementation groups account for 74.65% of all phishing websites. This shows that a small number of phishing kits is likely responsible for a significant proportion of sophisticated phishing websites in the wild. We discover that the Timing cloaking type has the most groups (394) among all cloaking types. Because this cloaking technique is less popular according to our findings, we suspect that prominent phishing kit developers do not deploy it, though individual criminals may still want to leverage it. Among the largest groups, we observe that one group of Click Through cloaking accounted for 16.45% (1,275) of code variants. As many as 18.67% (284) of the Notification Window occurrences were within a single group.

Implementation groups in the Public Dataset. We also compare the cloaking groups within the Public Dataset Guarnieri (2019), which was sourced from OpenPhish openphish (2014), PhishTank Phishtank (2006), PhishStats PhishStats (2020), and other phishing URL archives. Using this additional dataset, we can validate that the APWG dataset was representative of the ecosystem and evaluate the existence of other cloaking techniques that may not have been present in the APWG dataset. Table 3.3 shows detailed statistics on the cloaking group distributions between the two datasets. The number of groups found for each cloaking type from both datasets is similar. The Timing and Alert cloaking techniques still have the highest overall number of groups, which matches the findings from the APWG dataset. The number of groups for Click Through cloaking, however, increases from 51 to 105. We suspect that different phishers are developing more phishing kits with this cloaking technique because they realize that it can effectively evade detection by anti-phishing systems.

In addition, by comparing the AST similarity of implementation groups between the Public Dataset and the APWG Dataset, we discover that the same groups of cloaking technique types exist in both datasets. 11 out of 17 distinct groups of the Notification cloaking technique in the APWG Dataset also appear in the Public Dataset. Additionally, the Alert and Timing cloaking techniques have the most identical groups between the two datasets. This result indicates that phishing kits leveraging client-site cloaking techniques are widely used.

Evolution of cloaking groups over time. Because we crawled phishing data in both 2018 and 2019 from APWG feed, this dataset enables us to trace the origin of each cloaking type. The Timing, Alert, and Mouse Detection cloaking techniques were first used in phishing websites from June 2018 in our dataset. The (more advanced) Notification technique first appeared in November 2018. The early occurrence of these evasion methods reminds us that phishers are trying to stay one step ahead of the anti-phishing ecosystem. While re-

searchers and anti-phishing entities were working on mitigations against server-side cloaking techniques Invernizzi *et al.* (2016); Oest *et al.* (2018), those attackers had already turned their focus toward implementing client-side evasion methods. We suspect that those client-side cloaking techniques may have already been employed well before June 2018 Invernizzi *et al.* (2016); Kim *et al.* (2017) (the date we started crawling).

We also observe the evolution of cloaking techniques from the perspective of obfuscation. From our crawling process, we found that the code obfuscation rate on phishing websites increased from 20.79% in 2018 to 24.04% in 2019. For example, for the Pop-up cloaking technique, the earliest variant from June 2018 was not obfuscated. Gradually, phishers started to obfuscate their cloaking technique implementations: in October 2018, they added an encoding algorithm, while the AST structure remained highly similar to un-obfuscated implementations. Later, phishers started to symmetrically encrypt client-side cloaking techniques (e.g., by using AES) and included decryption keys only as request parameters. In such cases, the AST of the same cloaking technique would differ from an existing group, so we place them in a new group. However, with CrawlPhish, we still find similar web API calls, so we consider this group to be an evolution of a prior group (its *origin*). From this finding, we gain the intuition that cybercriminals are improving client-side cloaking techniques in phishing to make the latest implementations more difficult to analyze.

3.6.4 Trends in Cloaking Usage

Table 3.4 shows the prevalence of each client-side cloaking technique type that CrawlPhish detected. Note that the sum of each cloaking technique's occurrence may exceed 100% because some phishing websites implement multiple cloaking techniques. In the table, the percentage under the "2018", "2019", and "Total" columns represents the share of each *category* of JavaScript cloaking technique implementation in the respective time

Cloaking Technique		2018	2019	Total	Share
Category	Type	Count (%)	Count (%)	Count (%)	Count (%)
Fingerprinting	Cookie	1,295 (21.50%)	6,842 (23.56%)	8,137 (23.20%)	4,395 (12.53%)
	Referrer				2,346 (6.69%)
	User-Agent				1,396 (3.98%)
User Interaction	Pop-up: Alert	2,416 (40.11%)	17,782 (61.23%)	20,198 (57.60%)	6,027 (17.19%)
	Pop-up: Notification				1,521 (4.34%)
	Click Through				7,753 (22.11%)
	Mouse Detection				5,797 (16.53%)
Bot Behavior	Randomization	2,427 (40.29%)	6,141 (21.14%)	8,568 (24.43%)	1,623 (4.63%)
	Timing				6,945 (19.80%)
Total Cloaking Implementations		6,138	30,765	36,903	-

Table 3.4: Cloaking technique types in the APWG Dataset, as detected by CrawlPhish.

period. The percentage under the Share column refers to the percentage of each *type* of cloaking technique in all the cloaked phishing websites we detected.

We categorize the cloaking types in phishing websites from both the APWG Dataset and the Public Dataset. As shown in Table 3.4, the User Interaction cloaking category has the most implementations among phishing websites in the APWG Dataset. In 2018, 2,416 phishing websites (40.11%) leveraged cloaking within the User Interaction category, while in 2019, the usage ratio of User Interaction cloaking grew to 61.23%. The usage ratio of cloaking techniques in the Fingerprinting category over two years is almost the same. Within the Bot Behavior category, the usage ratio dropped significantly, from 40.29% to 21.14%. We find that phishing websites rely more on cloaking techniques in the User Interaction category than the others. We believe that this is because it is more difficult for anti-phishing crawlers to impersonate human behaviors than to bypass other types of cloaking.

Cloaking Technique		Total		Share	
Category	Type	Count	Percentage	Count	Percentage
Fingerprinting	Cookie	6,633	24.28%	2,912	9.87%
	Referrer			2,665	9.03%
	User-Agent			1,056	3.58%
User Interaction	Pop-up: Alert	17,634	64.55%	7,641	25.89%
	Pop-up: Notification			1,233	4.18%
	Click Through			6,735	22.82%
	Mouse Detection			2,025	6.86%
Bot Behavior	Randomization	5,294	19.38%	262	0.89%
	Timing			4,987	16.90%
Total Cloaking Implementations		29,561	-	-	-

Table 3.5: Cloaking technique types in the Public Dataset (September to December 2019), as detected by CrawlPhish.

Table 3.5 demonstrates the usage of each cloaking type CrawlPhish detected from the Public Dataset. Just as we observed from the 2019 portion of the APWG Dataset, the User Interaction category was also the most frequently implemented in the Public Dataset.

Brand distribution. Among the 6,024 cloaked phishing sites in 2018, LinkedIn and PayPal were the most frequently impersonated brands, as shown in Table 3.6. In 2019, the distribution changed: Apple and Bank of America phishing websites were the most prevalent. Overall, four of the top five brands in 2018 were also in the top five in 2019. Nevertheless, because of changes within the phishing landscape between the two years, our findings regarding the relative distribution of cloaking phishing websites may be skewed.

3.7 Evaluation: Impact of Cloaking techniques

We have, thus far, shown that phishing websites make extensive use of client-side cloaking techniques. To demonstrate that this cloaking represents a significant threat to users,

2018			2019		
Targeted Brand	**Count**	**Share**	**Targeted Brand**	**Count**	**Share**
LinkedIn	2,317	38.46%	Apple	6,298	21.69%
PayPal	1,104	18.33%	Bank of America	3,572	12.30%
Microsoft	646	10.72%	Facebook	2,230	7.68%
Bank of America	309	5.13%	PayPal	1,841	6.34%
Apple	153	2.54%	Microsoft	987	3.40%

Table 3.6: Top brands targeted by cloaked phishing websites in the APWG Dataset.

we deployed two experiments to verify that these techniques can truly evade detection by anti-phishing systems, and that they generally do not discourage victim visits—the two key factors to increasing attackers' return-on-investment.

3.7.1 Effectiveness Against Anti-Phishing Entities

We evaluate how effective client-side cloaking techniques are against real-world anti-phishing systems. Using a testbed for empirically measuring anti-phishing blacklists Oest *et al.* (2019), we first deployed 150 carefully-controlled artificial PayPal-branded phishing websites using new and previously unseen domain names: 50 for each of the top three User Interaction cloaking types we found in the wild (Notification, Click Through with a fake CAPTCHA, and Mouse Detection). We then simultaneously reported the URLs to key anti-phishing entities across the ecosystem (Google Safe Browsing, PhishTank, Netcraft, APWG, PayPal, and US CERT Oest *et al.* (2018)) to evaluate if the ecosystem can collectively detect our cloaked websites. Lastly, we monitored the detection status (i.e., blacklisting) of our websites in major web browsers (Google Chrome, Opera, and Microsoft Edge, each powered by different detection backends) over seven days.

At the conclusion of these experiments, we found that *none* of our phishing websites

were blacklisted in any browser, with the exception of Click Through websites, 21 (42%) of which were blocked in Microsoft Edge a median of 3 hours after we reported them. The detection occurred because Microsoft SmartScreen classified the obfuscation in the JavaScript source code as malware, not because it was capable of bypassing the cloaking technique itself. The fact that so many of our websites remained unmitigated after a seven-day period shows that client-side evasion methods are indeed effective at evading detection by modern anti-phishing systems.

Manual inspection is used by some anti-phishing entities Google (2020). Recurring suspicious websites that cannot be detected by automated systems should go to manual inspection for further analysis. With specialists' inspection, any malicious websites therein should be labeled as phishing and be blacklisted to protect users. Our observations, however, imply that our test phishing websites may have simply been classified as benign by anti-phishing systems and never sent for manual review. We believe that this is a clear limitation of current anti-phishing mitigations. Therefore, it is important for the whole anti-phishing ecosystem to understand the nature and prevalence of client-side cloaking techniques used by sophisticated phishing websites, especially when we consider the growth of such websites Oest *et al.* (2020c).

3.7.2 Hampering Victim User Traffic

To verify that client-side cloaking techniques in the User Interaction category do not significantly prevent users from being exposed to phishing content on cloaked phishing websites, we conducted an IRB-approved user study through Amazon Mechanical Turk Amazon (2005). Using a free hosting provider, we generated three websites: one with each of the same three types of cloaking as considered in the previous section (Notification, Click Through with a fake CAPTCHA, and Mouse Detection). Rather than hiding phishing content behind the cloaking, however, we simply hid the text "Hello World". By default, a

	Mouse Detection	**Click Through**	**Notification Window**
	Count (%)	Count (%)	Count (%)
Can See	879 (100.00%)	859 (97.72%)	374 (42.55%)
Cannot See	0 (0.00%)	20 (2.28%)	505 (57.45%)

Table 3.7: Experimental results on the effect of cloaking techniques on users' ability to see phishing content.

blank page would be shown. We then hired 1,000 workers in Amazon Mechanical Turk and requested them to report what they saw after visiting each of the three websites Hostinger (2014). We choose these three cloaking techniques because they are unique to client-side (rather than server-side) cloaking implementations, and because the other techniques have been tested in a server-side context Oest *et al.* (2019).

Table 3.7 shows the detailed experimental results. 121 of the 1,000 workers could not view our phishing websites due to a technical problem: their browsers automatically added "www" in front of the sub-domains in our URLs, which may occur in older versions of web browsers Club (2016). Thus, the responses of 879 workers were suitable for analysis.

For the Mouse Movement cloaking technique, 100% of the workers saw the "Hello World" text, and thus would have also seen phishing content had they visited a malicious website. For the Click Through websites, 97.72% saw the text, which shows that this cloaking technique is also effective against users. However, only 42.55% of the users saw the text on websites with the Notification Window cloaking technique. Nearly all users who did not see the text (94.94%) opted to *deny* notifications; the rest had incompatible browsers.

Although two of the cloaking techniques did not significantly prevent users from viewing the content, we found that the Notification Window cloaking technique has a negative

impact on phishing success rates against potential victims. However, had these users been successfully deceived by a phishing lure (e.g., one that conveys a sense of urgency) prior to visiting the page, we believe that they would have been more likely to allow notifications Van Der Heijden and Allodi (2019). Moreover, given the fact that websites with this cloaking technique were not detectable by the anti-phishing ecosystem (as we showed in section 3.7), we still believe that this technique remains viable overall. In fact, the website shown in Figure 3.4 was still online in January 2020 even though we first observed the phishing URL in May 2019.

Consequently, we conclude that client-side cloaking techniques in the User Interaction category enable phishing websites to maintain profitability through a much longer life span, generally without discouraging victim visits, which in turn allows phishers to harm more users.

3.7.3 Responsible Disclosure

Once we established that the cloaking techniques discovered by CrawlPhish were capable of evading anti-phishing systems while remaining effective against human victims, we disclosed our findings, and the corresponding JavaScript code for each technique tested, to the major anti-phishing blacklist operators: Google, Microsoft, and Opera. All companies acknowledged receipt of our disclosure. Google followed up by requesting more information on the semantics and prevalence of the cloaking techniques, and concurred with our finding that such techniques could potentially bypass detection by current automated anti-phishing systems.

3.8 Countering Client-side Cloaking Techniques

As we have observed, phishers make extensive use of sophisticated evasion techniques in their phishing attacks. The unique feature of client-side cloaking techniques is to re-

quire visitors to interact with the website or browser, such as through a button click or mouse movement. Phishers adopt such strategies because they believe that their victims will exhibit these behaviors when visiting a website Rotolo (2016). If the website is in the process of rendering and shows a blank page, most people tend to move their mouse subconsciously. Similarly, out of habit, users will click a button from a pop-up or notification window to make web page content appear. We expect that phishers' degree of sophistication will only continue to grow. Therefore, the ecosystem should ensure that existing detection and mitigation systems are capable of adapting to such evasion techniques.

To detect advanced phishing websites with client-side cloaking techniques, anti-phishing crawlers should match the behaviors that sophisticated phishing kits expect. Specifically, crawlers need to impersonate human behaviors such as mouse movement and button clicks. To examine a given website, anti-phishing systems can emulate such behaviors using automated browsers. In addition, as we observed in our analysis, the *Notification Window* technique seems to exploit the lack of support for web notifications by current automated browsers. Thus, it is important for anti-phishing systems to close this gap and ensure that the browsers being used for detection support the same features as those used by potential victims.

Also, CrawlPhish can be directly incorporated into existing anti-phishing crawlers. With the hidden web page content revealed by CrawlPhish alongside traditional attributes such as URLs, we believe that current anti-phishing systems could identify malicious websites that would otherwise evade detection. Furthermore, by implementing CrawlPhish analysis, crawlers would be able to more accurately classify and fingerprint new variants of evasion techniques employed phishing websites, or even discover entirely new types of cloaking. Such analysis would be particularly helpful in countering phishing websites that cannot currently be classified with high confidence.

Studies on phishing and cloaking techniques: Oest et al. analyzed server-side cloaking techniques within a dataset of 2,313 phishing kits and proposed a taxonomy of five different types of cloaking Oest *et al.* (2018). These authors also showed that cloaking techniques, including basic JavaScript cloaking, can effectively bypass detection by anti-phishing blacklists Oest *et al.* (2019). Based on an end-to-end analysis of large-scale phishing attacks, Oest et al. discovered that phishing websites with sophisticated evasion techniques are prevalent in the wild but the anti-phishing ecosystem has not effectively mitigated them Oest *et al.* (2020c). In this work, we have presented the first in-depth analysis of client-side cloaking techniques in the context of phishing based on a dataset of 112,005 live phishing websites.

Invernizzi et al. studied server-side web cloaking techniques against search engines, and proposed mechanisms to identify and bypass such cloaking Invernizzi *et al.* (2016). CrawlPhish leverages these methods to overcome server-side cloaking during crawling. The authors rooted their study in black markets and built a classifier to detect cloaking techniques implemented on the server side that returned different content to distinct browsing clients. This work mainly focused on the mutation of browser profiles to bypass server-side cloaking techniques to discover divergent web content. The authors found that 11.7% of search results were cloaked. The authors considered cloaking techniques used for Search Engine Optimization (SEO), advertisements, and drive-by download attacks. However, they did not investigate client-side cloaking techniques implemented in JavaScript (i.e., that execute in the browser). In contrast, we discovered diverse client-side cloaking techniques and analyzed them from the perspective of phishing attacks.

JavaScript analysis techniques: Although a number of static analysis Zhang *et al.* (2011b); Curtsinger *et al.* (2011); Kapravelos *et al.* (2013) and dynamic analysis Kolbitsch *et al.*

(2012); Kim *et al.* (2017) approaches have been proposed to analyze malicious JavaScript code, there has been no attempt to automatically extract JavaScript code semantics for identifying and classifying cloaking techniques. Arrow and Zozzle are static analysis methods to classify JavaScript malware based on previously discovered malicious scripts Zhang *et al.* (2011b); Curtsinger *et al.* (2011). Revolver tried to detect evasive JavaScript code through similarity checks against known malicious matters Kapravelos *et al.* (2013). Rozzle is a multi-execution virtual machine to explore multiple execution paths in parallel for enhancing the efficiency of dynamic analysis so that it can be used in large-scale experiments Kolbitsch *et al.* (2012). J-Force enhanced dynamic analysis methods to find hidden malicious behaviors by force-executing JavaScript code, regardless of the conditions, to explore all possible execution paths in an automated way Kim *et al.* (2017). Hence, J-Force lends itself well to revealing content hidden behind JavaScript cloaking code.

Analysis of program semantics similar to ours has been performed within other contexts. To deal with virtualization-based obfuscation, Coogan et al. proposed a de-obfuscation approach that identifies behaviors of malicious programs based on the flow of values to system calls Coogan *et al.* (2011). BEAGLE assigns semantics to malware by dynamically monitoring system and API calls that malware uses to compare versions of malicious code and quantify their differences—to observe the evolution of a series of malware Lindorfer *et al.* (2012). Zhang et al. introduced a semantic-based static analysis approach to reveal malicious Android applications' behaviors regardless of minor implementation differences Zhang *et al.* (2014). The authors leveraged an API dependency graph to determine the semantics of the program to classify malware and identify malware variants.

3.10 Conclusion

Through the first in-depth analysis of the client-side JavaScript code used by phishing websites, we have uncovered a wide gamut of sophisticated evasion techniques used by

attackers. In addition to categorizing such evasion techniques based on their semantics, our approach enabled us to measure the prevalence of each technique in the wild. In doing so, we observed that client-side evasion is becoming increasingly common.

Client-side JavaScript enables website developers to implement complex interactions between their websites and visitors. Thus, evasion techniques implemented in this manner pose a particular threat to the ecosystem: websites that use them can effectively discriminate between automated crawler visits and potential human victims. Unfortunately, client-side evasion techniques are difficult to analyze due to the dynamic nature of JavaScript code. CrawlPhish addresses this difficulty in a scalable manner. In addition to being able to detect and categorize client-side evasion with high accuracy, our approach can also track the origin of different implementations.

Given the rise of sophisticated phishing websites in the wild, we believe that automated analysis systems such as CrawlPhish are essential to maintaining an understanding of phishers' evolving tactics. Methodology such as ours can be incorporated by the ecosystem to more expeditiously and more reliably detect sophisticated phishing, which, in turn, can help prevent users from falling victim to these attacks through the continuous enhancement of the appropriate mitigations.

Chapter 4

REDIRECTION ABUSE IN PHISHING

To study the prevalence and nature of malicious redirection implemented in phishing attacks, first I conducted a preliminary analysis on a list of phishing URLs reported to one of the organizations that are most phishing-targeted. Through the analysis on the crawled redirection phishing data, 88% of phishing websites implement redirection technique, while less than 30% of phishing sites with redirection get blacklisted by Google Safe Browsing after three days of observation, which is a representative of blacklist based anti-phishing systems and which Internet users rely on most. With the crawled data and visualized redirect chains, we summarized several common features of malicious redirection. Among them, there are five typical attributes, length of redirect chain, number of shorted links, number of distinct geolocations, malicious server reuse, and client-side redirect method. To further contribute our findings on the improvement of the whole ecosystem, we quantify the attributes by comparing them in both benign and malicious redirection use. Lastly, with the quantified attributes, we propose recommendations for PayPal's anti-phishing system to improve the ability against phishing redirection.

4.1 Data Collection

The crawling process encounters with some obstacles. First of all, cloaking techniques may evade our crawler if they detects the IP from anti-phishing systems or a non-browser visit. Also, a non-browser crawling process cannot detect client side redirect type such as JavaScript and meta data. Therefore, we design our redirection crawler to stay in a residential IP and to open the suspicious URL in a real browser. With the help of a browser extension, the crawler can collect the IP address, URL, redirect type, and HTTP request

and response for each hop. Besides, the source code of the destination URL is crawled for further analysis.

During the internship in this summer, we have successfully crawled 35,814 distinct URLs from the data source of collaborated organization. 81% of them contain redirection, but only 29% of redirected URLs get blacklisted. The crawling result shows the prevalence and ability of redirection in phishing evading detections from anti-phishing systems.

4.2 Visualization

We visualize phishing websites implementing redirect chain that we have been crawling in Figure 4.1. The only blue dot represents the start point of all redirect chains, the yellow nodes refer to hops that are composed of redirect chains, and the red ones indicate that the hops are blacklisted by anti-phishing systems, Google Safe Browsing in our case. The figure reflects the ability of anti-phishing systems against phishing websites embedded with redirect chain. The less red dots shown in the figure, the less ability the anti-phishing system has against such sophisticated phishing attacks. The detection speed on phishing websites with redirection is truly slowed down, comparing with an average blacklist speed of eight hours according to Golden Hour Oest *et al.* (2020c). Specifically, there are two giant nodes on the top of Figure 4.1. These hops are dedicated servers that redirect traffic to the phishing websites from shortened links. Among over 500 outgoing edges, only 7 destinations were blacklisted, which indicates again that the current anti-phishing methodologies are not efficient against advanced phishing websites.

4.3 Underground Services

We discovered that in some phishing kits, redirection service has to be used when phishers attempts to deploy the phishing attack. Usually in a comment, the developer of this phishing kit clearly declare to turn redirection on via the dedicated server/URL he/she sets

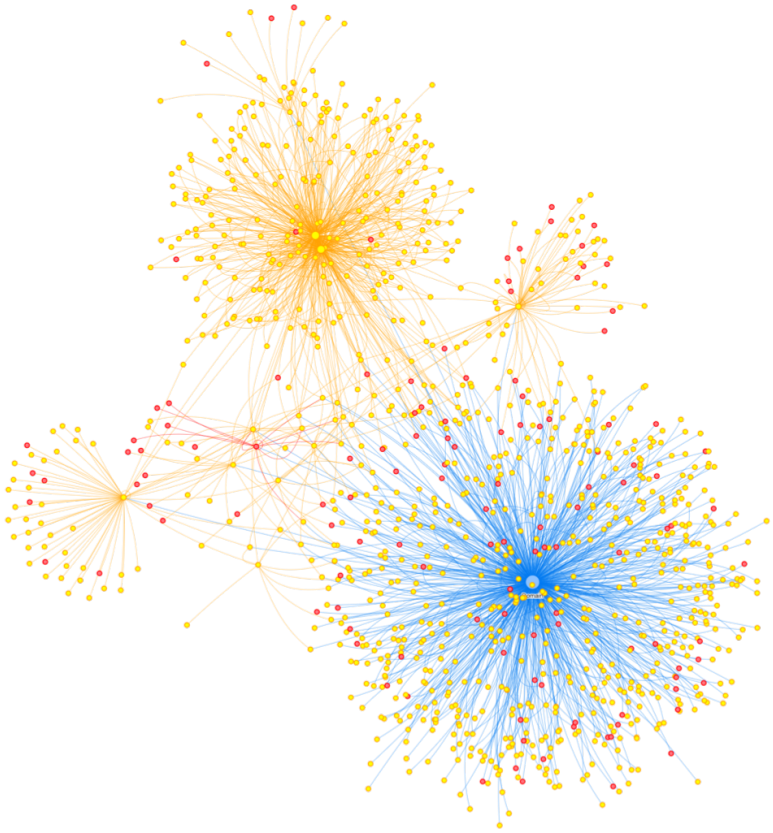


Figure 4.1: Visualization of phishing websites with redirect chain.

up, or the attack will not be deployed successfully.

As a side finding, .htaccess file exists in all phishing kits we have found. In this file, phishing servers deny requests from known IPs or User-Agents that are marked as anti-phishing entities or crawlers. It indicates that current phishing attacks have widely deployed IP/User-Agent cloaking techniques from server-side to evade detection from anti-phishing entities.

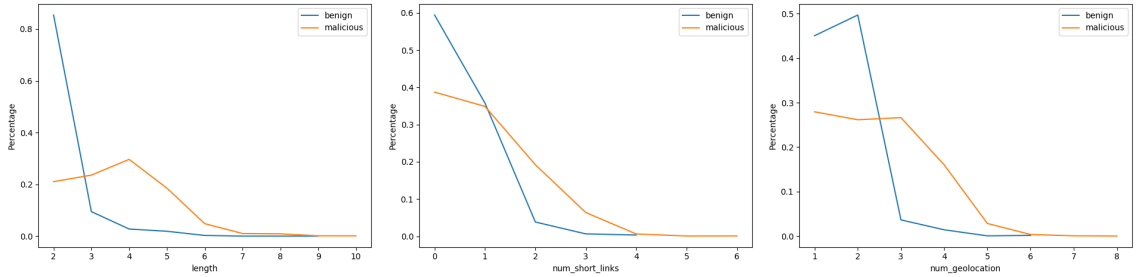
4.4 Features of Malicious Use of Redirection

By analyzing the redirection data we have been crawling since May 2020, we have summarized seven features that indicate the malicious use of redirection.

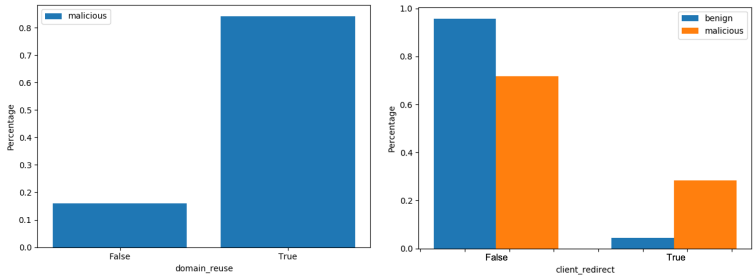
- long redirect chain
- multiple shortened links
- dedicated server (shared hops)
- heterogeneity of the IP/geo-locations
- redirect chain + a legit website as a destination (e.g., google, paypal) - cloaking may exists
- source code of destination website
- redirect type (JS indicates more malicious than server and meta)

To quantify the indicators for further implementation on the current anti-phishing systems, we compare those features between malicious and benign redirection use. Excitingly, the quantification result shows a clear border between benign malicious redirection. Typically, we have quantified five indicators, which are length of redirect chain, number of shortened links, number of geo-locations/IPs, dedicated server reuse, client/meta redirect type.

Figure 4.2 shows the comparisons between redirection implementations in phishing attacks and in legitimate websites in terms of indicators. From Figure 4.2a, we set the indicator as the length of redirect chain *greater than 3* because 94% of benign redirection have length within 3. Similarly in Figure 4.2b, 95% of benign redirection contains at most one shortened link service, while 73% of malicious use have no more than 1 shorted link services. Therefore, the attribute of number of shortened link services in the chain should be set as *greater than 1*. As for the indicator of distinct number of geolocations/IPs in the chain, we discovered that only 5% of benign redirection have more than 2 distinct IPs, but over 55% of malicious ones contain 3 or more different IPs. So a chain with



(a) Length of redirect chain. (b) Number of short links. (c) Number of geo-locations.



(d) Dedicated server reuse. (e) Client redirect type.

Figure 4.2: Comparisons between malicious and benign redirection implementation to quantify indicators.

greater than 2 different geolocations indicates malicious use of redirection. Because we did not observe any dedicated malicious server reuse in the benign redirection, if we find any existing server reuse in the redirect chain, the chain should be flagged out. Lastly, benign redirection seldom implements JavaScript or meta data redirect type in the chain, according to Figure 4.2e, and hence such redirect type often implies a malicious event.

4.5 Attribute Evaluation

We evaluated our attributes of malicious redirection to verify if they can identify maliciousness that the current ecosystem ignores. We first get 10,103 URLs that were marked as benign by an anti-phishing system and run our crawler to get the redirection information for each URL. Then we examine the URLs with the attributes of malicious redirection, which is weighted based on the ratio of attributes found in the malicious redirection dataset.

	Attributes (in a redirect chain)		
	Name	*Type*	*Explanation*
Redirect Type	301	Numerical	Number of Moved Permanently code
	302	Numerical	Number of Found code
	303	Numerical	Number of See Other code
	307	Numerical	Number of Temporary Redirect code
	308	Numerical	Number of Permanent Redirect code
	JavaScript	Numerical	Number of JavaScript redirects
	meta	Numerical	Number of meta redirects
Advanced Attribute	chain_length	Numerical	Length of the chain
	shortened_links	Numerical	Number of shortened links
	distinct_geo	Numerical	Number of distinct IP/geo-locations
Ecosystem	blacklisted	Categorial	Is any of hops blacklisted
	label	Categorial	Binary: benign or malicious

Table 4.1: Overview of redirection classifier attributes.

Among the list of 10,103 URLs, we discovered that 2,088 of them contain redirection. Within the redirected URLs, the attributes flag 567 malicious ones. To evaluate the accuracy of the attributes, we randomly selected half of malicious redirected URLs and manually check them. The false positive rate is 4.55%. With such result, we proved that the attributes proposed based on the analysis of redirection abuse in phishing can flag the phishing URLs that are ignored by the current anti-phishing systems.

4.6 Malicious Redirection Classification

I also implement machine learning algorithms to classify malicious redirection out of all redirection usage. According to the preliminary findings from section 4.5, the attributes of malicious redirection differ from those of benign redirections. Therefore, it is feasible and efficient that machine learning algorithms can perform on classification of a redirection.

We refine the features as shown in Table 4.1. We categorize the attributes into three

main groups, Redirect Type, Advanced Attribute, and Ecosystem. All features in Redirect Type group are how redirection performs in a redirect chain, either through server-side redirect method such as 30X status code or via client-side method such as JavaScript and meta data. Features in Advanced Attribute group represents those extracted and refined from a redirect chain, as discussed in section 4.4. In the last group, "blacklist" attribute indicates the ability of the anti-phishing ecosystem to detect such malicious redirect chain and "label" is used to label which class a redirect chain belongs to.

The current version of classifier is designed to meet the requirement of one of our collaborators, which requires a 0.5% False Positive (FP) rate. I conducted the classification experiments 20 times and calculated FP rate on validation set. The average FP rate is 0.66%. 12 of out 20 experiments have shown that our classifier satisfies the requirement of 0.5% FP rate. But outliers happened typically on Experiment 5, 7, 13, 16, and 20 mainly because of small validation dataset. I believe that with a larger dataset , the classifier will perform better from perspective of FP rate.

4.7 Related Work

Redirection has been implemented into spam for more than 10 years. Bhargrava et al. compares legitimate and malicious redirection and finds that server-side redirection is prominent on both benign and malicious use while spammers leverage client-side one more often than legitimate organizations Bhargrava *et al.* (2009). Chellapilla et al. studies the nature of JavaScript redirection spam techniques and indicates that obfuscation in JavaScript are very prevalent in evading the detection from static analysis based anti-spam system Chellapilla and Maykov (2007). The articles above show a direction on the analysis of redirection abuse.

Based on the taxonomy and nature analysis of redirection techniques, researchers began to consider redirection as a feature to classify malicious behaviors online. Webb et

al. proposes a methodology to collect redirection data automatically, including the framework, redirection data to collect, and preliminary analysis, which allows us to continue research on the modern redirection abuse Webb *et al.* (2006). Fette et al. detects phishing emails using several features, which includes whether shortened link is embedded in the email Fette *et al.* (2007). To make the outlook of emails clean and beautiful, legitimate senders often shorten a long URL. For example, Twitter often includes *t.co* when sending out emails to the users. Hence, the feature of shortened link inclusion cannot be used alone to identify maliciousness of an email. Other work focuses on the phishing detection on a specific area such as Twitter using the information of the account along with features of the tweets including redirection Aggarwal *et al.* (2012); Jeong *et al.* (2016); Lee and Kim (2013); Thomas *et al.* (2011); Grier *et al.* (2010). Because nowadays phishers often combine redirection with prevalent cloaking techniques in phishing attacks Oest *et al.* (2018, 2019, 2020a,d); Zhang *et al.* (2021), web page content of phishing websites might be difficult to acquire. Therefore, it becomes more critical that we need to leverage the redirection data without additional content information to classify the maliciousness of a website with redirection.

Others, on the other hand, work on the analysis of redirection. Kwon et al. converts and correlates redirect chains into a redirection graph and hence discovers that spammers like to reuse and share the intermediate redirect hops to maximize the return-on-investment rate and that different geo-locations of redirect hops and multiple landing URLs can also be helpful evading detection from anti-phishing systems Kwon *et al.* (2017). However, the work does not include the redirect type, such as server-side or client-side methods to implement redirection, and how the phishers deploy phishing with redirection, such as through shortened link services or intermediate hop reuse. The attributes mentioned above are very popular in redirection abuse, according to our preliminary findings, which can be much weighted ones in classifying malicious URLs.

Chapter 5

SPARTACUS: CLOAK USERS AGAINST CLOAKED PHISHING WEBSITES

According to the situation where cloaking techniques are widely implemented in phishing attacks and the goal of anti-phishing is to prevent Internet users to see phishing contents, we consider the security problem from a different perspective: instead of trying best to reveal phishing content, we can leverage the cloaking techniques in phishing to prevent users to see phishing content. In this work, we propose *Spartacus*, a framework that disguise Internet users as anti-phishing crawlers to request web page content of a suspicious URL, while remain users' own profiles on visiting whitelisted websites.

5.1 Design

Figure 5.1 demonstrates the architecture of the Spartacus system. The whole system consists of two parts, *front end* and *back end*. The front end is responsible for filtering whitelisted or blacklisted URLs and mutating profiles in the HTTP request such as user

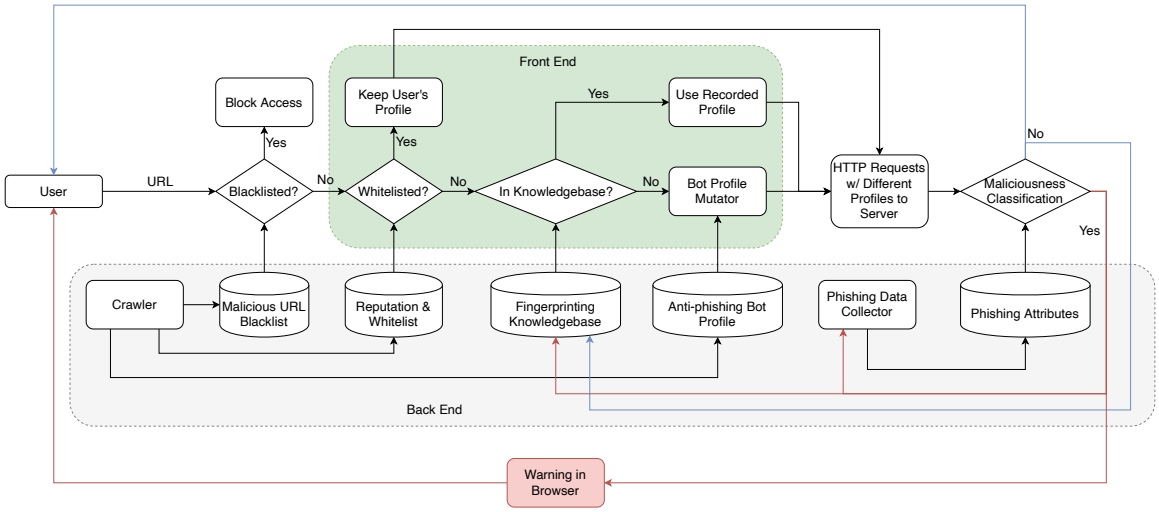


Figure 5.1: Spartacus architecture.

agent. In the back end, we maintain several databases to provide support to the front end, including black and whitelist database, which is used to process known URLs, fingerprinting knowledge base, which stores the successful mutation, the anti-phishing bot profile database, which contains the profiles of anti-phishing systems such as google bot, phishing attributes, which is used to distinguish the maliciousness of returned web page content.

When a user tries to visit a URL, first the URL goes to black and whitelist. If the existing record is found, the browser will either block the access or keep user's original profile when requesting the server. Otherwise, the front end will query the fingerprinting knowledgebase to see if we have processed the same url before, if found, we will use recorded profile to request the website, if not, we will mutate profile to a bot one. Then with the "spartacus" profile, HTTP request is sent to the server. After getting the page content, we determine whether there is maliciousness. If yes, the browser will show a warning sign to the user, or the web page content will be shown to the user. The knowledge base will be updated accordingly under both situations.

5.1.1 Collection and Maintenance of Back End Databases

Black- and whitelist database. Blacklist based anti-phishing systems such as Google Safe Browsing and Microsoft SmartScreen have been examining and collecting phishing URLs to warn users before they visit the websites. We collect known phishing URLs from common blacklists so that Spartacus can leverage the achievements from the anti-phishing systems to facilitate the web request process when dealing with known phishing websites. Similarly, we leverage a whitelist to avoid any impacts brought by Spartacus so that the users can visit legitimate websites with their own profiles.

Fingerprinting Knowledgebase. The fingerprinting knowledgebase is shared among users who leverage Spartacus and stores profiles with which users cannot see malicious content when requesting the server. After the maliciousness classification, if phishing content is

not found in the responded source, then we mark the mutated profile used in the request this time, connect it with the URL, and update the pair as success into the knowledgebase. When other users try to visit the same URL, Spartacus will use the corresponding profile recorded in the database to guarantee that users do not receive malicious content.

Anti-phishing Bot Profile Database. The profiles of anti-phishing system crawlers are keeping updating all the time. Accordingly, phishers refresh their blacklist to precisely evade the access from the anti-phishing bots. Therefore, we crawl the up-to-date profiles of anti-phishing systems into the database and hence Spartacus can leverage them to visit suspicious websites and receive benign web page content.

Phishing Attribute Database. After sending request with mutated profile and receiving respond from the server, Spartacus needs to classify whether maliciousness exists in the web page content. It implements the state-of-the-art phishing detections such as content-based and visual-similarity-based methodologies to extract phishing attributes based on the responded web page content such as screenshots and HTML source code. Spartacus compares such features with those from websites of most phishing-targeted organizations, which stores in the Phishing Attribute Datatbase.

5.1.2 Decision Makers on A URL Visit

When a user attempts to visit a URL, the URL will go through several decision makers to assure a safe browsing. The decision makers consist of Blacklist Filter, Whitelist Filter, Knowledgebase Query, and Maliciousness Classification. We will elaborate them in order of the system flow.

Blacklist Filter. With the contribution of the anti-phishing ecosystem, Spartacus can filter known phishing URLs through the query to the Malicious URL BlackList Database. Any match in the blacklist database will result in a block access to the URL without further action.

Whitelist Filter. Similar to the blacklist filter, the whitelist filter in Spartacus helps filter the whitelisted URLs so that the user can receive the web page content with best out-looking that the organization configures to fit in the visit from different browsers or devices.

Knowledgebase Query. When the URL does not fall into either Blacklist or Whitelist filter, Spartacus will examine whether such URL has been visited once by either the user or other users by querying the Fingerprinting Knowledgebase. If a record is found in the knowledgebase, Spartacus will mutate the web request profile according to the record to make sure that such web request profile will return a benign web page content; otherwise, Spartacus will call a *Bot Profile Mutator*, which randomly selects one profile from Anti-phishing Bot Profile Database to submit the web request and remains the impact to possible web page layout to the lowest.

Maliciousness Classification. After submitting the web request to the server, Spartacus along with the browser will receive a response from the server. There are different situations of the response as follows:

(1) The server responds either a benign page or an error page with status code of 4XX/5XX, or, it redirects the visit to a benign website, which is typically the website it impersonates;

(2) The server returns malicious web page content.

The former result is because (a) the server is malicious with evasion techniques and determines the visit from Spartacus as anti-phishing bot visit, or (b) the website is benign itself. The latter one is caused by (a) a malicious website without evasion techniques, or (b) a malicious website with evasion techniques, which is not triggered by the mutated profile from Spartacus.

The latter situation is what Spartacus needs to handle. As for reason (a), Spartacus relies on the current anti-phishing systems that are well capable of detecting simple phishing websites within a short time period and hence the whole ecosystem can be protected from

such phishing attacks. For reason (b), the phishing website implements evasion techniques that remain unknown after one visit. However, as Spartacus can be used by a large number of users, there is high possibility where a number of users will visit the same URL. Spartacus collects all attempts of mutations of web request profiles to such URL and records both successful and failed attempts in the Fingerprinting Knowledgebase so that we can fingerprint the trigger of the evasion technique in the phishing website and apply such configuration for all Spartacus users. Before Spartacus figures out the cloaking technique, it implements visual-similarity and content-based phishing detection mechanisms and alerts the user if any of the attributes detected in the content by matching the features with those in Phishing Attribute Database.

5.2 Related Work

Researchers have been studying phishing for several decades. They have proposed several methodologies to detect phishing attacks based on one or some of features from URL, content, and etc, and then warn users before visiting the deceptive websites. Some work analyzes URL of a suspicious website based on the lexical features or URL ranking to determine the maliciousness of the site Blum *et al.* (2010); Le *et al.* (2011); Khonji *et al.* (2012); Feroz and Mengel (2015). Others collect web page content and detect phishing websites with textual and visual similarity features Zhang *et al.* (2007, 2011a); Dunlop *et al.* (2010). Combining with available features including both URL and web page content, researchers have developed blacklist based anti-phishing systems such as Google Safe Browsing Whittaker *et al.* (2010) to protect Internet users from visiting suspicious websites. With the large-scale implementation of cloaking techniques in phishing attacks Oest *et al.* (2020d,a, 2019, 2018), researchers realize that the sophisticated phishing attacks are responsible for substantial portion of damage and that the whole ecosystem should prioritize on mitigating phishing with evasion techniques. Therefore, analysis and detection of

server-side Wang *et al.* (2011); Invernizzi *et al.* (2016) and client-side Zhang *et al.* (2021) cloaking techniques have been proposed to fight against such sophistications.

All the proposed methodologies, however, have a limitation that they only provide a pain-relief pills to certain symptoms of phishing attacks, such as suspicious URL, similar layout as the impersonated website, and cloaking techniques. Furthermore, as phishers have been developing advanced cloaking techniques, it becomes more and more difficult to retrieve phishing content, which most anti-phishing systems depend on. With very limited amount of features of a suspicious website, the system cannot determine precisely the maliciousness. Therefore, with the contribution of previous studies on phishing symptoms, our work provides a vaccine to neutralizing advanced phishing-virus. Our system is also extendable and always up-to-date by adding fingerprints that researchers will find in the future.

Chapter 6

COMPLETED AND PLANNED WORK

Completed Work. I have completed the following work proposed for my dissertation.

- *Systems And Methods For Detecting Phishing Websites.* It is a filed US patent with OCP.D2019.0577.US1/70481.2623US01.
- *CrawlPhish: Large-scale Analysis of Client-side Cloaking Techniques in Phishing.* It has been accepted and published in the Proceedings of the 42nd IEEE Symposium on Security & Privacy.
- *Neutralizing Evasion Techniques of Malicious Websites.* It is a filed US patent with OCP.D2020.101492.US1 (7879-16800).

Planned Work. Regarding to the redirection project discussed in chapter 4, I have completed the preliminary research including designing a crawler of phishing direction data, extracting features that are useful to identify malicious redirection, proposing heuristic redirection classification methodology, and reaching out organizations in the ecosystem to provide my findings. During the rest of my Ph.D. course, I will finish the design of a machine learning based redirection classifier and collaborate with other entities in the ecosystem to deploy my work to better detect advanced phishing websites with redirection and hence protect the whole anti-phishing ecosystem.

As for the novel anti-phishing system, Spartacus, I finished the design of the framework in format of a browser extension, as mentioned in chapter 5. Before my graduation, I will continue working on the deployment and evaluation of Spartacus. I will first evaluate the effectiveness of the new system by opening the reported advanced phishing URLs in the

real browser with Spartacus, recording the response resources including status code, source code, and screenshots, and comparing the result with that opened in a normal browser. I then will recruit volunteers to use Spartacus in their daily Internet activities and have them finish a survey on the convenience, effectiveness, and efficiency after several days of usage.

REFERENCES

Aggarwal, A., A. Rajadesingan and P. Kumaraguru, "Phishari: Automatic realtime phishing detection on twitter", in "2012 eCrime Researchers Summit", pp. 1–12 (IEEE, 2012).

Alowaisheq, E., P. Wang, S. Alrwais, X. Liao, X. Wang, T. Alowaisheq, X. Mi, S. Tang and B. Liu, "Cracking the wall of confinement: Understanding and analyzing malicious domain take-downs", in "Proceedings of the Network and Distributed System Security Symposium (NDSS)", (2019).

Amazon, "Amazon mechanical turk", https://www.mturk.com/ (2005).

Anderson, B., "Best automation testing tools for 2018 (top 10 reviews)", https://medium.com/@briananderson2209/best-automation-testing-tools-for-2018-top-10-reviews-8a4a19f664d2 (2017).

APWG, "Anti-Phishing Working Group: APWG Trends Report Q1 2018", URL https://docs.apwg.org/reports/apwg_trends_report_q1_2018.pdf, (Date last accessed 31-August-2018) (2018).

APWG, "Phishing Activity Trends Report 3rd Quarter 2019", https://docs.apwg.org/reports/apwg_trends_report_q3_2019.pdf (2019).

Bhargrava, K., D. Brewer and K. Li, "A study of url redirection indicating spam", CEAS (July 2009) (2009).

Bilge, L., E. Kirda, C. Kruegel and M. Balduzzi, "Exposure: Finding malicious domains using passive dns analysis.", in "Ndss", pp. 1–17 (2011).

Bin, S., W. Qiaoyan and L. Xiaoying, "A dns based anti-phishing approach", in "2010 Second International Conference on Networks Security, Wireless Communications and Trusted Computing", vol. 2, pp. 262–265 (IEEE, 2010).

Blum, A., B. Wardman, T. Solorio and G. Warner, "Lexical feature based phishing url detection using online learning", in "Proceedings of the 3rd ACM Workshop on Artificial Intelligence and Security", pp. 54–60 (ACM, 2010).

Borgolte, K., C. Kruegel and G. Vigna, "Meerkat: Detecting website defacements through image-based object recognition", in "24th USENIX Security Symposium USENIX Security 15)", pp. 595–610 (2015).

Canali, D., D. Balzarotti and A. Francillon, "The role of web hosting providers in detecting compromised websites", in "Proceedings of the 22nd International Conference on World Wide Web", WWW '13, pp. 177–188 (ACM, New York, NY, USA, 2013a), URL http://doi.acm.org/10.1145/2488388.2488405.

Canali, D., D. Balzarotti and A. Francillon, "The role of web hosting providers in detecting compromised websites", in "Proceedings of the 22nd international conference on World Wide Web", pp. 177–188 (ACM, 2013b).

Chellapilla, K. and A. Maykov, "A taxonomy of javascript redirection spam", in "Proceedings of the 3rd international workshop on Adversarial information retrieval on the web", pp. 81–88 (2007).

Club, T. W., "Web browser automatically adds www to url", https://www.thewindowsclub.com/browser-automatically-adds-www-to-url (2016).

Coogan, K., G. Lu and S. Debray, "Deobfuscation of virtualization-obfuscated software: a semantics-based approach", in "Proceedings of the 18th ACM conference on Computer and communications security", pp. 275–284 (2011).

Cova, M., C. Kruegel and G. Vigna, "Detection and analysis of drive-by-download attacks and malicious javascript code", in "Proceedings of the 19th international conference on World wide web", pp. 281–290 (2010).

Curtsinger, C., B. Livshits, B. G. Zorn and C. Seifert, "Zozzle: Fast and precise in-browser javascript malware detection.", in "USENIX Security Symposium", pp. 33–48 (San Francisco, 2011).

Dhamija, R., J. D. Tygar and M. Hearst, "Why phishing works", in "Proceedings of the SIGCHI conference on Human Factors in computing systems", pp. 581–590 (ACM, 2006).

Docs, M. W., "Mozilla web apis", https://developer.mozilla.org/en-US/docs/Web/API (2020).

Dunlop, M., S. Groat and D. Shelly, "Goldphish: Using images for content-based phishing analysis", in "2010 Fifth international conference on internet monitoring and protection", pp. 123–128 (IEEE, 2010).

Feroz, M. N. and S. Mengel, "Phishing url detection using url ranking", in "2015 ieee international congress on big data", pp. 635–638 (IEEE, 2015).

Fette, I., N. Sadeh and A. Tomasic, "Learning to detect phishing emails", in "Proceedings of the 16th international conference on World Wide Web", pp. 649–656 (ACM, 2007).

Fielding, R., J. Gettys, J. Mogul, H. Frystyk, L. Masinter, P. Leach and T. Berners-Lee, "Rfc2616: Hypertext transfer protocol–http/1.1", (1999).

Google, "Manual actions report", https://support.google.com/webmasters/answer/9044175?hl=en&ref_topic=4596795 (2020).

Grier, C., K. Thomas, V. Paxson and M. Zhang, "@spam: the underground on 140 characters or less", in "Proceedings of the 17th ACM conference on Computer and communications security", pp. 27–37 (2010).

Guarnieri, C., "The Year of the Phish", https://nex.sx/blog/212/15/the-year-of-the-phish.html (2019).

Guo, Z., "World-wide cloaking phishing websites detection", (2017).

Hamming, R. W., "Error detecting and error correcting codes", The Bell system technical journal **29**, 2, 147–160 (1950).

Holmes, A. and M. Kellogg, "Automating functional tests using selenium", in "AGILE 2006 (AGILE'06)", pp. 6–pp (IEEE, 2006).

Hostinger, "000webhost: Free web hosting", `https://www.000webhost.com/migrate?static=true` (2014).

Huang, H., L. Qian and Y. Wang, "A svm-based technique to detect phishing urls", Information Technology Journal **11**, 7, 921–925 (2012).

Invernizzi, L., K. Thomas, A. Kapravelos, O. Comanescu, J.-M. Picod and E. Bursztein, "Cloak of visibility: Detecting when machines browse a different web", in "2016 IEEE Symposium on Security and Privacy (SP)", pp. 743–758 (IEEE, 2016).

Jeong, S. Y., Y. S. Koh and G. Dobbie, "Phishing detection on twitter streams", in "Pacific-Asia Conference on Knowledge Discovery and Data Mining", pp. 141–153 (Springer, 2016).

Jules, D. S., "Jsinspect: Detect copy-pasted and structurally similar code", `https://github.com/danielstjules/jsinspect` (2017).

Kachalov, T. and zamotkin, "Javascript obfuscator", `https://github.com/javascript-obfuscator/javascript-obfuscator` (2016).

Kapravelos, A., Y. Shoshitaishvili, M. Cova, C. Kruegel and G. Vigna, "Revolver: An automated approach to the detection of evasive web-based malware", in "Presented as part of the 22nd USENIX Security Symposium", pp. 637–652 (2013).

Katalon, "Katalon studio", `https://www.katalon.com/katalon-studio/` (2015).

Khonji, M., Y. Iraqi and A. Jones, "Enhancing phishing e-mail classifiers: A lexical url analysis approach", International Journal for Information Security Research (IJISR) **2**, 1/2, 40 (2012).

Khonji, M., A. Jones and Y. Iraqi, "A novel phishing classification based on url features", in "2011 IEEE GCC conference and exhibition (GCC)", pp. 221–224 (IEEE, 2011).

Kim, K., I. L. Kim, C. H. Kim, Y. Kwon, Y. Zheng, X. Zhang and D. Xu, "J-force: Forced execution on javascript", in "Proceedings of the 26th international conference on World Wide Web", pp. 897–906 (International World Wide Web Conferences Steering Committee, 2017).

Kolbitsch, C., B. Livshits, B. Zorn and C. Seifert, "Rozzle: De-cloaking internet malware", in "2012 IEEE Symposium on Security and Privacy", pp. 443–457 (IEEE, 2012).

Kwon, H., M. B. Baig and L. Akoglu, "A domain-agnostic approach to spam-url detection via redirects", in "Pacific-Asia Conference on Knowledge Discovery and Data Mining", pp. 220–232 (Springer, 2017).

Le, A., A. Markopoulou and M. Faloutsos, "Phishdef: Url names say it all", in "2011 Proceedings IEEE INFOCOM", pp. 191–195 (IEEE, 2011).

Lee, S. and J. Kim, "Warningbird: A near real-time detection system for suspicious urls in twitter stream", IEEE transactions on dependable and secure computing **10**, 3, 183–195 (2013).

Leontiadis, N., T. Moore and N. Christin, "Measuring and analyzing search-redirection attacks in the illicit online prescription drug trade.", in "USENIX Security Symposium", vol. 11 (2011).

Liang, B., M. Su, W. You, W. Shi and G. Yang, "Cracking classifiers for evasion: a case study on the google's phishing pages filter", in "Proceedings of the 25th International Conference on World Wide Web", pp. 345–356 (2016).

Lindorfer, M., A. Di Federico, F. Maggi, P. M. Comparetti and S. Zanero, "Lines of malicious code: insights into the malicious software industry", in "Proceedings of the 28th Annual Computer Security Applications Conference", pp. 349–358 (2012).

Microsoft, "Windows defender smartscreen", `https://github.com/MicrosoftDocs/windows-itpro-docs/blob/public/windows/security/threat-protection/windows-defender-smartscreen/windows-defender-smartscreen-overview.md` (2019).

Modi, A., Z. Sun, A. Panwar, T. Khairnar, Z. Zhao, A. Doupé, G.-J. Ahn and P. Black, "Towards automated threat intelligence fusion", in "2016 IEEE 2nd International Conference on Collaboration and Internet Computing (CIC)", pp. 408–416 (IEEE, 2016).

Mozilla, "Event reference", `https://developer.mozilla.org/en-US/docs/Web/Events` (2020).

Niu, X.-m. and Y.-h. Jiao, "An overview of perceptual hashing", Acta Electronica Sinica **36**, 7, 1405–1411 (2008).

Oest, A., Y. Safaei, A. Doupé, G.-J. Ahn, B. Wardman and K. Tyers, "Phishfarm: A scalable framework for measuring the effectiveness of evasion techniques against browser phishing blacklists", in "Proceedings of the 40th IEEE Symposium on Security and Privacy (Oakland)", pp. 764–781 (Oakland, CA, 2019).

Oest, A., Y. Safaei, A. Doupé, G.-J. Ahn, B. Wardman and G. Warner, "Inside a phisher's mind: Understanding the anti-phishing ecosystem through phishing kit analysis", in "2018 APWG Symposium on Electronic Crime Research (eCrime)", pp. 1–12 (IEEE, 2018).

Oest, A., Y. Safaei, P. Zhang, B. Wardman, K. Tyers, Y. Shoshitaishvili and A. Doupé, "Phishtime: Continuous longitudinal measurement of the effectiveness of anti-phishing blacklists", in "29th {USENIX} Security Symposium ({USENIX} Security 20)", pp. 379–396 (2020a).

Oest, A., Y. Safaei, P. Zhang, B. Wardman, K. Tyers, Y. Shoshitaishvili, A. Doupé and G.-J. Ahn, "PhishTime: Continuous longitudinal measurement of the effectiveness of anti-phishing blacklists", in "Proceedings of the 29th USENIX Security Symposium", (2020b).

Oest, A., P. Zhang, B. Wardman, E. Nunes, J. Burgis, A. Zand, K. Thomas, A. Doupé and G.-J. Ahn, "Sunrise to sunset: Analyzing the end-to-end life cycle and effectiveness of phishing attacks at scale", in "Proceedings of the 29th USENIX Security Symposium", (2020c).

Oest, A., P. Zhang, B. Wardman, E. Nunes, J. Burgis, A. Zand, K. Thomas, A. Doupé and G.-J. Ahn, "Sunrise to sunset: Analyzing the end-to-end life cycle and effectiveness of phishing attacks at scale", in "29th {USENIX} Security Symposium ({USENIX} Security 20)", (2020d).

openphish, "OpenPhish", https://openphish.com (2014).

Paya, I. C. and T. Chow, "Combining a browser cache and cookies to improve the security of token-based authentication protocols", US Patent 7,240,192 (2007).

PhishStats, "PhishStats", `https://phishstats.info/` (2020).

Phishtank, "PhishTank", `https://phishtank.com` (2006).

Rotolo, T., "Mouse movement patterns and user frustration", `https://www.trymyui.com/blog/2016/10/28/mouse-movement-patterns-and-user-frustration/` (2016).

Sood, A. K. and R. J. Enbody, "Crimeware-as-a-service: a survey of commoditized crimeware in the underground market", International Journal of Critical Infrastructure Protection **6**, 1, 28–38 (2013).

Soska, K. and N. Christin, "Automatically detecting vulnerable websites before they turn malicious", in "23rd {USENIX} Security Symposium ({USENIX} Security 14)", pp. 625–640 (2014).

Stockley, M., "How scammers abuse google search's open redirect feature", `https://nakedsecurity.sophos.com/2020/05/15/how-scammers-abuse-google-searchs-open-redirect-feature/` (2020).

Stone-Gross, B., T. Holz, G. Stringhini and G. Vigna, "The underground economy of spam: A botmaster's perspective of coordinating large-scale spam campaigns.", LEET **11**, 4–4 (2011).

Sun, Z., C. E. Rubio-Medrano, Z. Zhao, T. Bao, A. Doupé and G.-J. Ahn, "Understanding and predicting private interactions in underground forums", in "Proceedings of the Ninth ACM Conference on Data and Application Security and Privacy (CODASPY)", (ACM, 2019).

Thomas, K., C. Grier, D. Song and V. Paxson, "Suspended accounts in retrospect: an analysis of twitter spam", in "Proceedings of the 2011 ACM SIGCOMM conference on Internet measurement conference", pp. 243–258 (2011).

Thomas, K., F. Li, A. Zand, J. Barrett, J. Ranieri, L. Invernizzi, Y. Markov, O. Comanescu, V. Eranti, A. Moscicki *et al.*, "Data breaches, phishing, or malware?: Understanding the risks of stolen credentials", in "Proceedings of the 2017 ACM SIGSAC conference on computer and communications security", pp. 1421–1434 (ACM, 2017).

Van Der Heijden, A. and L. Allodi, "Cognitive triaging of phishing attacks", in "28th USENIX Security Symposium", pp. 1309–1326 (2019).

Von Ahn, L., B. Maurer, C. McMillen, D. Abraham and M. Blum, "recaptcha: Human-based character recognition via web security measures", Science **321**, 5895, 1465–1468 (2008).

W3C, "Web notifications", `https://www.w3.org/TR/notifications/` (2015).

Wang, D. Y., S. Savage and G. M. Voelker, "Cloak and dagger: dynamics of web search cloaking", in "Proceedings of the 18th ACM Conference on Computer and Communications Security (CCS)", pp. 477–490 (ACM, 2011).

Wang, Y.-M. and M. Ma, "Detecting stealth web pages that use click-through cloaking", in "Microsoft Research Technical Report, MSR-TR", (2006).

Wardman, B., D. Clemens, J. Wolnski, P. Inc-San Jose, S. D.-U. States and S. D.-U. States, "Phorecasting phishing attacks: A new approach for predicting the appearance of phishing websites", Cyber-Security and Digital Forensics p. 142 (2016).

Webb, S., J. Caverlee and C. Pu, "Introducing the webb spam corpus: Using email spam to identify web spam automatically.", in "CEAS", (2006).

Whittaker, C., B. Ryner and M. Nazif, "Large-scale automatic classification of phishing pages", in "Proceedings of the 28th Network and Distributed System Security Symposium (NDSS)", (2010).

Wu, M., R. C. Miller and G. Little, "Web wallet: preventing phishing attacks by revealing user intentions", in "Proceedings of the second symposium on Usable privacy and security", pp. 102–113 (ACM, 2006).

Zhang, H., G. Liu, T. W. Chow and W. Liu, "Textual and visual content-based anti-phishing: a bayesian approach", IEEE Transactions on Neural Networks **22**, 10, 1532–1546 (2011a).

Zhang, J., C. Seifert, J. W. Stokes and W. Lee, "Arrow: Generating signatures to detect drive-by downloads", in "Proceedings of the 20th international conference on World wide web", pp. 187–196 (ACM, 2011b).

Zhang, M., Y. Duan, H. Yin and Z. Zhao, "Semantics-aware android malware classification using weighted contextual api dependency graphs", in "Proceedings of the 2014 ACM SIGSAC conference on computer and communications security", pp. 1105–1116 (2014).

Zhang, P., A. Oest, H. Cho, Z. Sun, R. Johnson, B. Wardman, S. Sarker, A. Kpravelos, T. Bao, R. Wang, Y. Shoshitaishvili, A. Doupé and G.-J. Ahn, "CrawlPhish: Large-scale Analysis of Client-side Cloaking Techniques in Phishing", in "Proceedings of the 42nd IEEE Symposium on Security and Privacy (Oakland)", (San Francisco, CA, 2021).

Zhang, Y., J. I. Hong and L. F. Cranor, "Cantina: a content-based approach to detecting phishing web sites", in "Proceedings of the 16th international conference on World Wide Web", pp. 639–648 (ACM, 2007).